

SpectraLang

Do Zero ao Avancado

Um tutorial completo da linguagem SpectraLang -- dos fundamentos da programacao ate tipos algebricos, tensores, async/await, APIs HTTP nativas e o compilador. Escrito como um livro-texto para quem esta começando do zero absoluto.

Linguagem compilada implementada em Rust · JIT/AOT via Cranelift · Memoria hibrida (GC + manual) ·
Seguranca de tipos rigorosa · Tensors, autodiff, async/await, HTTP nativo

CLI v0.3.0 · Licenca MIT · Julho 2026

Sumario

Sumario	2
Capitulo 1 -- O Que E Programacao -- E O Que E SpectraLang?	6
Tres Principios de Design	7
Como Este Livro Esta Organizado	8
Capitulo 2 -- Seu Primeiro Programa	9
Regras Estruturais de Todo Arquivo	10
Convencoes de Nomenclatura	11
Capitulo 3 -- Variaveis e Memoria	12
Os Tipos Primitivos	12
Declarando e Usando Variaveis	13
F-Strings: Interpolacao de Texto	14
Capitulo 4 -- Numeros e Aritmetica	15
Operadores Aritmeticos	15
Operadores de Comparacao e Logicos	16
Atribuicao Composta e Precedencia	16
Capitulo 5 -- Texto, Strings e Entrada/Saida	18
Manipulacao de Strings com std.string	18
std.char -- Operacoes em Caracteres	19
Capitulo 6 -- Tomando Decisoões -- if, else, e unless	20
Capitulo 7 -- Repetindo Coisas -- Loops	22
switch -- Selecao por Valor	23
Capitulo 8 -- Funcoes -- Organizando o Codigo	24
Visibilidade e Funcoes como Valores	25
Capitulo 9 -- Arrays, Tuplas e Intervalos	26
Capitulo 10 -- Structs e Metodos	28
Capitulo 11 -- Enums e Pattern Matching	30
Capitulo 12 -- Generics -- Codigo Que Funciona com Qualquer Tipo	32

Capitulo 13 -- Traits -- Definindo Comportamento Compartilhado	34
Capitulo 14 -- Closures e Funcoes de Alta Ordem	36
Capitulo 15 -- Option, Result, e o Operador ?	38
Capitulo 16 -- Sistema de Modulos e Visibilidade	40
Capitulo 17 -- std.io, std.string, std.char, std.math, std.convert	42
std.io -- Entrada e Saida	42
std.string -- Manipulacao de Strings	42
std.char -- Operacoes em Caracteres	43
std.math -- Funcoes Matematicas	43
std.convert -- Conversao de Tipos	44
Capitulo 18 -- std.collections, std.random, std.fs, std.env, std.time	45
std.collections -- Listas Dinamicas	45
std.random, std.fs, std.env	45
std.time -- Tempo e Duracao	46
Capitulo 19 -- Async/Await -- Concorrenca de Primeira Classe	47
Concorrenca com std.concurrent	48
Capitulo 20 -- Tensors e Autodiff	49
Tipos Tensor Estaticos	49
API de Criacao e Metadados	49
Autodiff: Blocos diff e backward	50
Capitulo 21 -- std.ml -- Machine Learning	51
Funcionalidades de ML	51
Capitulo 22 -- spectra.api -- HTTP, Routing, Middleware, e Database	52
Hello HTTP -- Servidor Minimo	52
Routing, Middleware, e JSON	52
Database Drivers	53
Capitulo 23 -- O Pipeline de Compilacao	54
As Quatro Crates Centrais	54
Capitulo 24 -- Front-end: Lexer, Parser e Semantica	55
Lexer	55

Parser	55
Análise Semântica Multipassada	55
Capítulo 25 -- Mid-end, Back-end e Runtime	56
IR SSA (SIR)	56
Otimização	56
Back-end Cranelift e Runtime	56
Capítulo 26 -- Toolchain: CLI, Package Manager, LSP	57
O Binário spectralang	57
Package Manager	57
LSP e Ferramentas de Desenvolvimento	57
Capítulo 27 -- Performance, Maturidade e Roadmap	58
Benchmarks Cross-Linguagem	58
Maturidade das Features da Linguagem	58
Capítulo 28 -- Apêndice A: Referência Rápida	60
Palavras-chave (39)	60
Precedência de Operadores	60
Visibilidade	60
Formas de import	60
Erros Comuns	61
Capítulo 29 -- Apêndice B: Gramática e Mapa de Arquivos	62
Gramática Informal (EBNF)	62
Mapa de Arquivos do Repositório	62
Capítulo 30 -- Apêndice C: Stdlib Completa e Roadmap	64
Todos os Módulos stdlib	64
Flags do CLI -- Resumo Completo	64
Fases do Roadmap	65
Documentos de Apoio no Repositório	66
Fim do Guia	66

PARTE - CAPITULO 1

O Que E Programacao -- E O Que E SpectraLang?

Capitulo 1 -- O Que E Programacao -- E O Que E SpectraLang?

Um computador, no fundo, e uma maquina extraordinariamente obediente e extraordinariamente estúpida. Ele executa instrucoes -- uma apos a outra, bilhoes de vezes por segundo -- sem jamais questionar o que esta fazendo. A unica coisa que ele entende sao numeros: codigos binarios que representam operacoes como 'some estes dois valores' ou 'guarde este numero na posicao de memoria 83742'. Escrever programas diretamente nessa linguagem de numeros -- chamada *codigo de maquina* -- e possivel, mas seria como tentar escrever um romance apontando para caracteres individuais numa tabela ASCII. Fazivel, mas tao lento e propenso a erros que ninguem faria.

E aqui que entra a ideia central de uma **linguagem de programacao**: ela e uma ponte entre a forma como nos, humanos, pensamos sobre problemas, e a forma como o computador executa instrucoes. Em vez de escrevermos sequencias de numeros binarios, escrevemos algo como `idade = 25;` -- uma frase que, para nos, tem significado imediato: 'crie um espaco na memoria chamado idade e coloque o valor 25 dentro dele'. Um programa especial chamado **compilador** le esse texto, verifica se ele faz sentido, e o traduz para o codigo de maquina que o processador entende. A linguagem de programacao e, portanto, um contrato: ela nos da um vocabulario e uma gramatica que podemos usar para descrever computacoes, e em troca o compilador garante que essas descricoes sejam traduzidas corretamente.

SpectraLang e uma dessas linguagens. Mas ela nao foi criada para ser 'mais uma linguagem'. Ela nasceu de uma observacao concreta: no mundo real, as pessoas que treinam modelos de inteligencia artificial usam Python, e as pessoas que colocam esses modelos em producao usam Go, Rust ou Java. Existe uma fronteira invisivel entre o mundo do treinamento e o mundo do *serving*, e cruza-la significa reescrever codigo, gerenciar dois runtimes diferentes, serializar modelos e conviver com custos de integracao que nao agregam valor algum ao produto final. A tese central de SpectraLang e que essa fronteira nao deveria existir: a mesma linguagem que descreve um tensor e sua retropropagacao deveria ser capaz de descrever a requisicao HTTP que serve o modelo treinado. Treino e serving, no mesmo arquivo, no mesmo runtime, no mesmo artefato de deploy.

Para realizar essa visao, SpectraLang foi construida sobre dois pilares complementares. O primeiro e o workstream de **IA/ML**: tensors com tipagem estatica de rank, dimensoes e

dispositivo; diferenciacao automatica integrada ao compilador; um framework completo de treinamento com modulos, otimizadores, datasets e dataloaders; exportacao ONNX; tokenizacao; e recuperacao aumentada por geracao (RAG). O segundo e o workstream de **API**, materializado no pacote `spectra.api`: um servidor HTTP nativo com tipos de primeira classe para Request, Response, Method, Status e Header; roteamento com parametros tipados; middleware composto; serializacao JSON via derive macros; e drivers para SQLite, PostgreSQL e Redis. O resultado e uma linguagem onde voce pode escrever um modelo de ML e o endpoint que o serve no mesmo arquivo, com o mesmo sistema de tipos verificando ambos.

SpectralLang e implementada em Rust como um workspace de multiplos crates. O compilador e dividido em quatro camadas: o **front-end** (lexer, parser, analise semantica e lint), o **mid-end** (rebaixamento para uma representacao intermediaria em forma SSA, otimizacoes, grafo de tensores e autodiff), o **back-end** (geracao de codigo via Cranelift, tanto em modo JIT -- compilacao sob demanda em memoria -- quanto AOT -- arquivos objeto nativos), e o **runtime** (gerenciamento hibrido de memoria com GC de rastreamento e alocao manual, reactor de plataforma para async/await, e a biblioteca padrao). Esta arquitetura em camadas significa que ferramentas como o LSP podem reutilizar o front-end diretamente, sem carregar Cranelift ou o runtime.

Tres Principios de Design

Toda linguagem de programacao faz escolhas. Quando o compilador encontra uma ambiguidade -- 'este codigo poderia significar duas coisas diferentes' -- alguem precisa decidir qual interpretacao prevalece. SpectraLang segue tres principios que guiam todas essas decisoes.

O primeiro principio e **clareza antes de concisao**. Muitas linguagens permitem omitir informacoes que o compilador 'poderia deduzir'. SpectraLang prefere que voce seja explicito. Se uma funcao retorna um valor, o tipo desse retorno deve estar declarado. Se uma variavel pertence a um tipo especifico, melhor declara-lo. A economia de alguns caracteres de digitacao nunca compensa a perda de clareza quando outra pessoa -- ou voce mesmo, seis meses depois -- tenta entender o que o codigo faz.

O segundo principio e **seguranca de tipos rigorosa**. SpectraLang pertence a familia das linguagens estaticamente tipadas: o compilador conhece o tipo de cada valor *antes* de o programa rodar, e rejeita operacoes que nao fazem sentido para esses tipos. Voce nao pode somar um numero com um texto, nao pode passar uma string onde se espera um inteiro, e -- com uma unica excecao -- nao pode misturar tipos numericos diferentes sem conversao explicita. A excecao e o alargamento de `int` para `float`, que e seguro porque todo inteiro cabe num float sem perda de precisao para valores ate 2^{53} . Essa rigidez nao e burocracia: e uma garantia de que categorias inteiras de bugs simplesmente nao podem existir nos seus programas.

O terceiro principio e **desempenho sem cerimonia**. Por padrao, SpectraLang gerencia a memoria para voce usando um coletor de lixo (GC) que rastreia quais valores ainda estao em uso e libera automaticamente os que nao estao mais. Para a vasta maioria do codigo -- logica de negocio, servidores web, scripts de treinamento -- isso e exatamente o que voce quer: voce cria valores, usa-os, e o runtime cuida do resto. Mas SpectraLang tambem oferece alocao manual

para momentos em que voce precisa de controle absoluto sobre o layout de memoria, como em kernels numericos de alta performance ou integracao com bibliotecas C. Essa dualidade -- GC para conveniencia, alocao manual para controle -- e o que chamamos de modelo de **memoria hibrida**.

Como Este Livro Esta Organizado

Este livro foi escrito para acompanhar voce desde o primeiro contato com programacao ate o dominio dos recursos mais avancados da linguagem. Cada capitulo introduz um novo conceito com explicacoes em prosa antes de mostrar codigo -- porque entender o 'porque' e mais importante do que memorizar o 'como'. Voce pode ler os capitulos em sequencia, como um curso completo, ou pular para secoes especificas quando precisar de referencia sobre um topico. As Partes I a V cobrem os fundamentos da linguagem: variaveis, controle de fluxo, funcoes, tipos compostos e abstracoes. As Partes VI a VIII exploram topicos avancados: tratamento de erros, sistema de modulos, concorrencia com `async/await`, tensores e machine learning. As Partes IX e X cobrem a plataforma de API e a toolchain. Os Apendices servem como referencia rapida. Seja qual for o seu ponto de partida, o objetivo e o mesmo: que voce termine este livro nao apenas sabendo *como* escrever SpectraLang, mas entendendo *por que* a linguagem funciona da forma como funciona.

PARTE - CAPITULO 2

Seu Primeiro Programa

Capitulo 2 -- Seu Primeiro Programa

Existe uma tradicao em programacao que remonta a 1978, quando Brian Kernighan e Dennis Ritchie publicaram o livro que definiu a linguagem C. O primeiro programa de todo iniciante imprime as palavras 'hello, world' na tela. A tradicao persiste porque ela encapsula, em poucas linhas, a essencia do que significa programar: voce escreve instrucoes, o computador as executa, e algo acontece no mundo -- neste caso, texto aparece no terminal. Vamos começar exatamente por ai.

Antes de escrever qualquer codigo, voce precisa de uma ferramenta capaz de transformar o que voce digita em algo que o computador execute. Spectralang oferece um binario chamado `spectralang` que funciona como porta de entrada para tudo. O comando mais importante para começar é `spectralang run`, que recebe um arquivo, compila-o sob demanda e executa o resultado imediatamente. Para instalar, voce precisa do Rust (versao 1.75 ou superior) instalado no seu sistema. Clone o repositório, compile e estará pronto:

Instalacao

```
1  git clone <repositorio>;
2  cd Spectralang
3  cargo build --release
4  ./target/release/spectralang --help
```

Agora crie um arquivo chamado `hello.spectra` com o seguinte conteudo. Vamos examinar cada linha, cada caractere, para entender exatamente o que está acontecendo:

hello.spectra

```
1  module hello;
2
3      import { println } from std.io;
4
5      pub fn main() -&gt; int {
6          println("Hello, World!");
7          return 0;
8      }
```

Fonte: `hello.spectra`. Execute com: `spectralang run hello.spectra`

A primeira linha, `module hello;`, declara que este arquivo pertence ao modulo chamado `hello`. Em Spectralang, todo arquivo deve começar com uma declaracao de modulo -- e a forma como a linguagem organiza o codigo em unidades com nome. Pense nos modulos como capitulos de um livro: cada um tem um titulo, e juntos eles formam a obra completa. O nome do modulo pode usar pontos para indicar hierarquia, como `module app.utils;`, mas para nosso primeiro

programa um nome simples e suficiente. A declaracao de modulo deve ser a primeira linha do arquivo -- comentarios sao permitidos antes dela.

A segunda linha, `import { println } from std.io;`, e uma instrucao de importacao. Ela diz ao compilador: 'eu quero usar a funcao chamada `println` que vive dentro do modulo `std.io`'. O prefixo `std` indica a biblioteca padrao da linguagem, e `io` e o submodulo que contem funcoes de entrada e saida -- como imprimir texto na tela e ler o que o usuario digita. Sem esta linha de importacao, o compilador nao saberia o que significa `println` e rejeitaria o programa com um erro. Importacoes sao a forma como voce declara quais ferramentas externas ao seu modulo pretende usar.

As linhas tres a seis formam o que chamamos de **funcao principal** ou **entry point**. A assinatura e `pub fn main() -> int`. Vamos decodificar cada parte: `pub` significa 'publico' -- esta funcao pode ser chamada de fora do modulo, o que e essencial porque o runtime precisa encontra-la para iniciar o programa. `fn` e a palavra-chave que declara uma funcao. `main` e o nome da funcao -- por convencao, o runtime sempre procura uma funcao com este nome para comecar a execucao. Os parenteses vazios `()` indicam que a funcao nao recebe nenhum parametro. A seta `-> int` declara que esta funcao retorna um valor do tipo `int` (um numero inteiro). O bloco entre chaves `{ }` contem o corpo da funcao.

Dentro do corpo, duas instrucoes. A primeira, `println("Hello, World!");`, chama a funcao `println` com um argumento: a string `"Hello, World!"`. `println` (do ingles 'print line') faz exatamente o que seu nome sugere: imprime o texto recebido e adiciona uma quebra de linha ao final. O ponto e virgula no final e obrigatorio em SpectraLang -- ele marca o fim de uma instrucao, assim como o ponto final marca o fim de uma frase. A segunda instrucao, `return 0;`, termina a funcao e devolve o valor 0 para quem a chamou (neste caso, o sistema operacional). Retornar 0 e uma convencao universal em programacao que significa 'tudo ocorreu bem'; qualquer outro valor indicaria que algo deu errado.

Execute o programa com `spectralang run hello.spectra` e voce vera `Hello, World!` aparecer no terminal. O que aconteceu nos bastidores? O front-end do compilador leu seu texto fonte e o transformou em uma arvore sintatica (AST). O mid-end converteu essa arvore em uma representacao intermediaria (IR) em forma SSA. O back-end Cranelift gerou codigo de maquina nativo para seu processador diretamente na memoria. O runtime inicializou o coletor de lixo, registrou as funcoes da biblioteca padrao, e transferiu o controle para sua funcao `main`. Tudo isso em milissegundos. Voce acabou de escrever, compilar e executar seu primeiro programa em SpectraLang.

Regras Estruturais de Todo Arquivo

Existem algumas regras que o compilador exige e que voce seguira em todos os programas SpectraLang que escrever. Elas nao sao arbitrariedades -- cada uma serve a um proposito de clareza, seguranca ou organizacao.

Primeiro, a declaracao de modulo deve ser a primeira linha de codigo do arquivo (comentarios podem aparecer antes). O nome do modulo pode usar pontos separando hierarquias. Segundo, importacoes devem vir logo apos a declaracao de modulo, antes de qualquer definicao. O compilador processa as importacoes durante a fase de resolucao semantica, e voce nao pode

usar algo que ainda nao foi importado. Terceiro, a funcao pub `fn main() -> int` e o ponto de entrada. O runtime a chama automaticamente; sem ela, nao ha por onde comecar. Quarto, instrucoes dentro de funcoes devem ser separadas por ponto e virgula, e blocos sao delimitados por chaves.

Convencoes de Nomenclatura

SpectralLang adota convencoes que ajudam a distinguir visualmente diferentes categorias de nomes no codigo. Variaveis e funcoes usam `snake_case` (minusculas com underscores, como `minha_variavel`). Tipos nomeados -- structs, enums e traits -- usam `PascalCase` (cada palavra comeca com maiuscula, como `Ponto` ou `RetanguloColorido`). Constantes seguem `SCREAMING_SNAKE_CASE` (todas maiusculas). Modulos usam `snake_case` com pontos como separadores. Essas convencoes nao sao impostas pelo compilador, mas segui-las torna o codigo imediatamente mais legivel para qualquer pessoa familiarizada com a linguagem.

PARTE - CAPITULO 3

Variaveis e Memoria

Capitulo 3 -- Variaveis e Memoria

No capitulo anterior, escrevemos um programa que imprime uma mensagem fixa. Mas programas de verdade precisam lembrar de coisas. Um servidor web precisa lembrar qual usuario fez login. Um jogo precisa lembrar a posicao do personagem. Uma rede neural precisa lembrar os pesos de cada conexao. Em programacao, 'lembrar de algo' significa armazenar um valor num local nomeado da memoria do computador. Esse local nomeado e o que chamamos de **variavel**.

Para entender variaveis, ajuda pensar na memoria do computador como um armazem gigantesco com bilhoes de gavetas numeradas. Cada gaveta (cada *byte* de memoria) tem um endereco unico. Quando voce declara uma variavel com `let idade = 25;`, o compilador reserva um conjunto de gavetas contiguas (8 bytes, no caso de um inteiro de 64 bits), grava o valor 25 dentro delas, e associa o nome `idade` a esse endereco. A partir desse momento, toda vez que voce escrever `idade` no codigo, o compilador sabe que deve ir ate aquelas gavetas especificas e ler ou escrever o valor. Voce nunca precisa saber o numero do endereco -- esse e o trabalho do compilador. Voce so usa o nome.

Mas o compilador precisa saber mais do que apenas o endereco. Ele precisa saber o **tipo** do valor armazenado. O tipo determina tres coisas fundamentais: quantos bytes a variavel ocupa, como interpretar os bits armazenados (os mesmos 8 bytes podem representar um inteiro, um float, ou parte de uma string, dependendo do tipo), e quais operacoes sao validas (voce pode multiplicar dois inteiros, mas nao faz sentido multiplicar duas strings). Em SpectraLang, o tipo pode ser declarado explicitamente -- `let contador: int = 0;` -- ou inferido pelo compilador a partir do valor inicial -- `let nome = "Alice";` deduz que `nome` e do tipo string. Nos dois casos, uma vez definido, o tipo da variavel nunca muda.

Diferente de linguagens como Rust, que exigem que voce declare explicitamente se uma variavel pode ser modificada, SpectraLang trata toda variavel como reatribuivel por padrao. Se voce escrever `let x = 10;` e depois `x = 20;`, a segunda linha simplesmente substitui o valor armazenado. A palavra-chave `mut` existe na gramatica e e aceita pelo parser, mas a semantica atual da linguagem nao distingue entre variaveis mutaveis e imutaveis no nivel local. Esta decisao de design prioriza a ergonomia: a maioria dos programas modifica variaveis com frequencia, e forcar anotacoes de mutabilidade em cada uma adicionaria ruido visual sem beneficio proporcional de segurancia.

Os Tipos Primitivos

SpectraLang oferece seis tipos primitivos -- os tijolos fundamentais com os quais todos os tipos mais complexos são construídos.

`int` é o tipo para números inteiros com sinal, armazenado em 64 bits. Pode representar valores de aproximadamente -9 quinquilhões a +9 quinquilhões. Você pode escrever literais inteiros com underscores para legibilidade: `1_000_000` e um milhão. `float` é o tipo para números com parte fracionária, seguindo o padrão IEEE 754 de 64 bits (dupla precisão). Isso significa que ele pode representar aproximadamente 15-17 dígitos significativos de precisão, mas com as mesmas limitações que todo float tem: números como 0.1 não podem ser representados exatamente em binário. `bool` é o tipo mais simples: só pode ser `true` ou `false`. Ocupa 1 byte e é o tipo que resulta de todas as operações de comparação.

`string` representa uma sequência de caracteres Unicode codificada em UTF-8, escritas entre aspas duplas. Sequências de escape permitem caracteres especiais: `\n` e quebra de linha, `\t` e tabulação, `\"` e uma aspa dupla literal. `char` é o tipo para um único caractere Unicode, escrito entre aspas simples: `'A'`, `'ä'`, `'.'`. Internamente, um `char` é representado como um inteiro contendo o código Unicode do caractere. Finalmente, `unit` é um tipo especial que representa 'nenhum valor' -- é o que uma função retorna quando não declara tipo de retorno, análogo ao `void` de C ou Java.

Uma característica importante de SpectraLang é sua política de conversão de tipos. A linguagem é extremamente restritiva quanto a conversões implícitas: a única conversão que acontece automaticamente é `int` para `float`. Qualquer outra conversão deve ser feita explicitamente usando funções do módulo `std.convert` ou o operador `as`. Esta restrição pode parecer pedante no início, mas ela previne uma classe inteira de bugs sutis que afligem linguagens mais permissivas, onde uma conversão automática inesperada produz resultados corretos em tipo mas errados em lógica de negócio.

Declarando e Usando Variáveis

```
1  let x = 10;                // int (inferido)
2  let pi = 3.14;             // float
3  let ativo = true;          // bool
4  let nome = "Alice";        // string
5  let letra = 'A';           // char
6
7  let contador: int = 0;      // tipo explícito
8  contador = contador + 1;    // reatribuição funciona
9
10 let arr = [10, 20, 30];
11 arr[0] = 99;                // modificação indexada OK
```

Declaração com `let`. O tipo é inferido ou anotado explicitamente.

Observe que a palavra-chave `let` introduz uma nova variável, mesmo que já exista outra com o mesmo nome em um escopo externo. Isso é chamado de **shadowing** (sombreamento): a nova variável 'esconde' a anterior dentro do escopo onde foi declarada. O linter de SpectraLang tem uma regra específica para detectar sombreamento, porque embora seja uma ferramenta útil em certas situações, o sombreamento acidental pode causar confusão. Se você quiser apenas modificar o valor de uma variável existente, não use `let` novamente -- apenas atribua o novo valor: `x = 42;`

F-Strings: Interpolacao de Texto

Imprimir valores de variaveis e uma das operacoes mais comuns em programacao -- seja para depurar um problema, exibir resultados para o usuario, ou gerar logs. SpectraLang oferece um mecanismo elegante chamado **f-strings** (formatted strings) que permite embutir expressoes diretamente dentro de uma string. A sintaxe e simples: prefixe a string com **f** e coloque expressoes entre chaves **{ }**. O compilador avalia cada expressao, converte o resultado para string, e o insere na posicao correspondente.

O que torna f-strings poderosas e que o conteudo entre chaves e uma expressao completa da linguagem -- voce pode colocar variaveis, operacoes aritmeticas, chamadas de funcao, ou acessos a campos de struct. O compilador valida a sintaxe e os tipos dentro das chaves como faria em qualquer outro lugar do codigo, entao voce nao precisa se preocupar com erros de formatacao que so apareceriam em tempo de execucao:

```
1  let nome = "Alice";
2      let idade = 30;
3      println(f'Ola, {nome}!');           // Ola, Alice!
4      println(f'Daqui a 5 anos: {idade + 5}'); // Daqui a 5 anos: 35
5
6      let x = 4;
7      println(f'O quadrado de {x} e {x * x}'); // O quadrado de 4 e 16
8
9      let p = Ponto { x: 10, y: 20 };
10     println(f'Posicao: ({p.x}, {p.y})');    // Posicao: (10, 20)
```

PARTE - CAPITULO 4

Numeros e Aritmetica

Capitulo 4 -- Numeros e Aritmetica

A aritmetica e a forma mais fundamental de computacao. Antes de existirem interfaces graficas, antes da internet, antes mesmo dos primeiros compiladores, os computadores foram inventados para fazer contas -- trajetorias balisticas, tabelas de logaritmos, calculos de engenharia. Hoje, a aritmetica continua sendo o alicerce de tudo que um computador faz: renderizar um frame de video e aritmetica matricial; aplicar um filtro numa imagem e aritmetica sobre pixels; treinar uma rede neural sao bilhoes de multiplicacoes e somas. Entender como a aritmetica funciona em SpectraLang -- as operacoes disponiveis, as regras de tipo, as armadilhas a evitar -- e essencial antes de avançar para qualquer topico mais complexo.

SpectraLang oferece os operadores aritmeticos que voce esperaria de qualquer linguagem, com duas particularidades importantes. A primeira e a regra de tipo para divisao: quando voce divide dois inteiros, o resultado e um inteiro -- a parte fracionaria e simplesmente descartada (a operacao *trunca* em direcao a zero). Isso significa que $10 / 3$ resulta em 3, nao em 3.333. Para obter divisao com parte decimal, pelo menos um dos operandos deve ser float. A segunda particularidade e que o operador de modulo % tambem funciona com floats, nao apenas com inteiros -- embora o uso mais comum seja com inteiros, onde ele retorna o resto da divisao inteira.

Existe tambem uma questao de precisao que todo programador precisa entender desde cedo: floats nao sao numeros reais. Eles sao aproximacoes com precisao finita. O numero 0.1, por exemplo, nao pode ser representado exatamente em binario (assim como $1/3$ nao pode ser representado exatamente em decimal -- e 0.333... infinito). Quando voce soma 0.1 dez vezes, o resultado nao e exatamente 1.0, mas algo como 0.9999999999999999. Isso nao e um bug de SpectraLang -- e uma consequencia inevitavel da representacao IEEE 754, compartilhada por praticamente todas as linguagens modernas. Para comparacoes entre floats, evite igualdade exata ==; prefira verificar se a diferenca entre os valores e menor que uma tolerancia (um epsilon). Para calculos financeiros que exigem exatidao decimal, use inteiros representando centavos, nao floats representando reais.

Operadores Aritmeticos

Operador	Operacao	Tipos	Exemplo
+	Adicao	int, float, string+string	3+4 -> 7
-	Subtracao	int, float	10-3 -> 7
*	Multiplicacao	int, float	3*4 -> 12

Operador	Operacao	Tipos	Exemplo
/	Divisao	int, float	10/3 -> 3 (int, trunca)
%	Modulo	int, float	10%3 -> 1

```

1  let soma = 10 + 5;           // 15
2  let dif = 10 - 3;           // 7
3  let prod = 4 * 3;           // 12
4  let quoc = 10 / 3;          // 3 (divisao inteira -- trunca)
5  let resto = 10 % 3;         // 1
6  let quoc_f = 10.0 / 3.0;    // 3.333... (float)
7  let neg = -(5 + 2);         // -7

```

Operadores aritmeticos. Atencao a diferenca entre divisao int e float.

Operadores de Comparacao e Logicos

Comparacoes sao operacoes que produzem um valor booleano (true ou false) a partir de dois operandos. Elas sao a base de toda tomada de decisao em programacao -- e atraves de comparacoes que um programa decide qual caminho seguir. Os seis operadores testam igualdade (==), diferenca (!=), e as quatro relacoes de ordem (<, >, <=, >=).

Operadores logicos combinam valores booleanos. && (E logico) retorna true apenas se ambos os operandos forem true. || (OU logico) retorna true se pelo menos um for true. ! (NAO logico) inverte o valor. Uma propriedade importante e a **avaliacao de curto-circuito**: o segundo operando so e avaliado se o primeiro nao for suficiente para determinar o resultado. Em a && b, se a for false, b nunca e avaliado. Isso nao e apenas uma otimizacao -- e uma garantia semantica que permite padroes como x != 0 && 100 / x > 5, onde a divisao so ocorre se x for diferente de zero.

```

1  let a = true; let b = false;
2  let e_logico = a && b;           // false (ambos precisam ser true)
3  let ou_logico = a || b;        // true (pelo menos um true)
4  let negacao = !a;              // false (inverte)
5  let complexo = (10 > 5) && (3 < 8); // true
6
7  // Curto-circuito na pratica
8  let divisor = 0;
9  let seguro = divisor != 0 && (100 / divisor) > 10; // false, sem divisao

```

Atribuicao Composta e Precedencia

SpectralLang suporta cinco operadores de atribuicao composta: +=, -=, *=, /=, e %=. E importante notar que, em SpectralLang, a atribuicao **nao e uma expressao** -- ela nao retorna valor. Voce nao pode escrever let y = (x = 10); como faria em C. Isso e uma decisao deliberada de design que evita uma classe de bugs sutis onde uma atribuicao acidental dentro de uma condicao (if x = 5 em vez de if x == 5) passa despercebida.

Nivel	Operadores
1 (alto)	() f() x.y x[i]
2	! (unario) - (unario)

Nivel	Operadores
3	* / %
4	+ -
5	< > <= >=
6	== !=
7	&&
8 (baixo)	

Divisao entre dois int e divisao inteira (trunca em direcao a zero). Para divisao decimal, use float em pelo menos um dos operandos.

PARTE - CAPITULO 5

Texto, Strings e Entrada/Saida

Capitulo 5 -- Texto, Strings e Entrada/Saida

Se a aritmetica e o musculo da computacao, o texto e sua pele -- a interface atraves da qual programas se comunicam com humanos. Tudo que voce le numa tela, desde o resultado de um comando ate uma pagina web completa, e texto em algum nivel de abstracao. SpectraLang trata texto como um cidadao de primeira classe: strings nao sao arrays de bytes disfarcados, mas sequencias Unicode com operacoes dedicadas e seguras.

O modulo `std.io` fornece as funcoes basicas de comunicacao com o mundo exterior. A dupla mais usada e `print` e `println`. A diferenca entre elas e sutil mas importante: `print` emite o texto e mantem o cursor na mesma linha; `println` emite o texto e move o cursor para a linha seguinte. Use `print` quando quiser construir uma linha em partes, e `println` quando cada mensagem deve ocupar sua propria linha. Existem tambem as variantes `eprint` e `eprintln` que escrevem na saida de erro padrao (`stderr`) em vez da saida normal (`stdout`) -- uma distincao importante em programas serios, onde mensagens de erro nao devem contaminar a saida de dados que pode estar sendo redirecionada para um arquivo ou outro programa.

Para ler dados do usuario, use `read_line()` (le uma linha completa) ou `input(prompt)` (exibe um prompt e retorna o que o usuario digitar). Ambas retornam strings -- se voce precisar de um numero, use as funcoes de conversao de `std.convert`. O exemplo a seguir demonstra o ciclo completo de entrada, processamento e saida:

```
1  import std.io;
2  import std.convert;
3
4  fn main() -&gt; int {
5      print("Digite seu nome: ");
6      let nome = read_line();
7
8      print("Digite sua idade: ");
9      let entrada = read_line();
10     let idade = std.convert.string_to_int(entrada);
11
12     println(f"Ola, {nome}!");
13     println(f"Daqui a 10 anos voce tera {idade + 10} anos.");
14     return 0;
15 }
```

Programa interativo simples. Entrada, conversao, saida.

Manipulacao de Strings com `std.string`

SpectralLang inclui um conjunto rico de funcoes para manipular strings no modulo `std.string`. Estas funcoes cobrem desde operacoes basicas como obter o comprimento da string e verificar se ela contem uma substring, ate transformacoes como converter para maiusculas/minusculas, remover espacos das pontas, e extrair fatias. Todas as operacoes respeitam a codificacao UTF-8 e operam sobre caracteres Unicode, nao sobre bytes -- embora a funcao `len` retorne o numero de bytes (nao de caracteres), o que e a convencao em linguagens de sistemas por razoes de performance.

Uma operacao particularmente util e `substring(s, inicio, fim)`, que extrai uma parte da string. Os indices seguem a convencao de intervalo semiaberto: `inicio` e inclusivo e `fim` e exclusivo. Funcoes como `split_first` e `split_last` permitem dividir strings por um separador sem alocar colecoes intermediarias. Para busca, `index_of` retorna a posicao (em bytes) da primeira ocorrencia, ou `-1` se nao encontrar. Para substituicao, `replace` substitui todas as ocorrencias de um padrao por outro.

Funcao	Assinatura	Descricao
<code>len</code>	<code>(string) -> int</code>	Numero de bytes
<code>contains</code>	<code>(string, string) -> bool</code>	Contem substring?
<code>to_upper</code>	<code>(string) -> string</code>	Converte ASCII para maiusculo
<code>to_lower</code>	<code>(string) -> string</code>	Converte ASCII para minusculo
<code>trim</code>	<code>(string) -> string</code>	Remove espacos das pontas
<code>starts_with</code>	<code>(s, prefix) -> bool</code>	Comeca com prefixo?
<code>ends_with</code>	<code>(s, suffix) -> bool</code>	Termina com sufixo?
<code>concat</code>	<code>(a, b) -> string</code>	Concatena duas strings
<code>char_at</code>	<code>(s, idx) -> int</code>	Codigo Unicode na posicao; <code>-1</code> se invalido
<code>substring</code>	<code>(s, start, end) -> string</code>	Extrai <code>[start, end)</code>
<code>replace</code>	<code>(s, from, to) -> string</code>	Substitui todas as ocorrencias
<code>index_of</code>	<code>(s, sub) -> int</code>	Primeira posicao; <code>-1</code> se nao encontrar
<code>reverse_str</code>	<code>(string) -> string</code>	Inverte a string

std.char -- Operacoes em Caracteres

Enquanto `std.string` opera sobre strings inteiras, `std.char` fornece operacoes sobre caracteres individuais. Essas funcoes recebem `int` (o codigo Unicode do caractere), nao valores do tipo `char`. Para usar essas funcoes com caracteres de uma string, voce primeiro obtem o codigo via `std.string.char_at()`, e entao passa esse inteiro para as funcoes de `std.char`. As funcoes incluem classificacao (`is_alpha`, `is_digit_char`, `is_whitespace_char`) e transformacao (`to_upper_char`, `to_lower_char`).

PARTE - CAPITULO 6

Tomando Decisoes -- if, else, e unless

Capitulo 6 -- Tomando Decisoes -- if, else, e unless

Ate agora, todos os nossos programas executaram instrucoes em linha reta: primeira linha, segunda linha, terceira linha, fim. Mas programas de verdade raramente sao lineares. Um servidor web precisa responder diferentemente a requisicoes GET e POST. Um jogo precisa verificar se o jogador colidiu com um obstaculo. Um programa de linha de comando precisa validar se os argumentos fazem sentido antes de prosseguir. Em todos esses casos, o programa precisa **tomar decisoes** -- executar um bloco de codigo ou outro, dependendo de uma condicao.

O mecanismo fundamental para tomada de decisao em SpectraLang e a construcao `if`. Sua estrutura e intuitiva: a palavra-chave `if` e seguida por uma condicao booleana e um bloco de codigo entre chaves. Se a condicao for verdadeira, o bloco e executado. Se for falsa, ele e ignorado. Opcionalmente, voce pode encadear multiplas condicoes com `elif` (a forma `elseif` tambem e aceita como alias), e fornecer um bloco `else` que e executado quando nenhuma das condicoes anteriores e verdadeira.

Mas `if` em SpectraLang tem uma caracteristica que o distingue de muitas outras linguagens: ele e uma **expressao**, nao apenas uma instrucao. Isso significa que um bloco `if` pode produzir um valor, que pode ser atribuido a uma variavel ou usado diretamente em outra expressao. Cada ramo deve retornar o mesmo tipo, e o ultimo valor de cada bloco (sem ponto e virgula) e o valor que o ramo produz. Isso elimina a necessidade do operador ternario (`cond ? a : b`) presente em linguagens como C e Java -- em SpectraLang, o proprio `if` cumpre esse papel de forma mais legivel e com verificacao de tipos completa.

SpectraLang tambem oferece a construcao `unless`, que e o inverso logico do `if`: o bloco e executado quando a condicao e **falsa**. Embora semanticamente equivalente a `if !condicao`, o `unless` pode tornar o codigo mais natural de ler em situacoes onde a intencao e 'faca algo a menos que esta condicao seja verdadeira' -- como em guardas de validacao: 'a menos que o usuario esteja autenticado, negue o acesso'.

```
1  let nota = 75;
2    if nota >= 90 { println("A"); }
3    elif nota >= 80 { println("B"); }
4    elif nota >= 70 { println("C"); }
5    else { println("F"); }
6
7    // if como expressao -- atribuindo o resultado a uma variavel
8    let classificacao = if nota >= 60 { "Aprovado" } else { "Reprovado" };
9    let max = if a > b { a } else { b };
10
11   // unless: executa quando a condicao e FALSA
12   let autenticado = false;
13   unless autenticado {
14     println("Acesso negado!");
15   }
```

PARTE - CAPITULO 7

Repetindo Coisas -- Loops

Capitulo 7 -- Repetindo Coisas -- Loops

Tomar decisoes e o primeiro passo para criar programas nao-lineares. O segundo passo e **repetir** acoes. Pense em quantas tarefas de programacao envolvem repeticao: processar cada linha de um arquivo, cada registro de um banco de dados, cada frame de uma animacao, cada epoca de um treinamento. Fazer isso manualmente -- copiar e colar o mesmo codigo centenas de vezes -- nao e apenas inviavel, e impossivel quando o numero de iteracoes nao e conhecido antecipadamente. Para resolver isso, toda linguagem oferece construcoes de **loop**.

SpectralLang oferece quatro formas de loop, cada uma adequada a um cenario diferente. O `while` repete enquanto uma condicao for verdadeira -- e verifica a condicao *antes* de cada iteracao, o que significa que o corpo pode nunca executar se a condicao ja for falsa no inicio. O `do-while` inverte essa logica: executa o corpo primeiro e verifica a condicao depois, garantindo pelo menos uma execucao. O `for...in` itera sobre intervalos ou colecoes, sendo a forma mais comum e idiomatica de loop em SpectralLang. Finalmente, o `loop` cria um laco infinito que so termina quando encontra um `break` explicito -- util para servidores e situacoes onde a condicao de saida e complexa demais para caber na declaracao do loop.

Duas palavras-chave controlam o fluxo dentro de qualquer loop. `break` interrompe o loop imediatamente e transfere o controle para a primeira instrucao apos o bloco. `continue` pula o restante da iteracao atual e avanca para a proxima verificacao da condicao. Usadas com criterio, essas construcoes permitem expressar logicas de repeticao complexas de forma clara e concisa:

```

1  // while -- verifica antes de executar
2      let i = 0;
3      while i < 5 {
4          println(f"i={i}");
5          i = i + 1;
6      }
7
8  // do-while -- executa pelo menos uma vez
9      let j = 0;
10     do {
11         println(f"j={j}");
12         j = j + 1;
13     } while j < 3;
14
15     // for..in -- itera sobre intervalos
16     for i in 0..5 { println(f"{i}"); } // 0,1,2,3,4 (exclusivo)
17     for i in 1..=5 { println(f"{i}"); } // 1,2,3,4,5 (inclusivo)
18
19     // loop -- infinito ate break
20     let cont = 0;
21     loop {
22         cont = cont + 1;
23         if cont >= 5 { break; }
24     }
25
26     // break e continue
27     for i in 0..10 {
28         if i == 3 { continue; } // pula i=3
29         if i == 7 { break; }    // sai do loop
30         println(f"{i}");      // imprime 0,1,2,4,5,6
31     }

```

Quatro formas de loop. `for..in` e a mais idiomática para intervalos conhecidos.

switch -- Selecao por Valor

Alem das construcoes baseadas em condicoes booleanas, SpectraLang oferece `switch` para selecao baseada em valores especificos. Enquanto `if/elif` funciona bem para condicoes arbitrariedades, `switch` e mais expressivo quando voce quer comparar uma unica variavel contra multiplos valores conhecidos. A sintaxe usa `case valor => bloco` para cada caso, e exige ou um caso `else` padrao ou cobertura exaustiva de todos os valores possiveis. O compilador verifica que voce nao esqueceu nenhum caso.

```

1  let dia = 3;
2      switch dia {
3          case 1 => { println("Segunda"); }
4          case 2 => { println("Terca"); }
5          case 3 => { println("Quarta"); }
6          else => { println("Outro dia"); }
7      }

```

PARTE - CAPÍTULO 8

Funcoes -- Organizando oCodigo

Capitulo 8 -- Funcoes -- Organizando oCodigo

Escrevemos varios programas ate agora, mas todos eles colocavam todo o codigo dentro da funcao `main`. Isso funciona para exemplos pequenos, mas se torna insustentavel rapidamente. Imagine um programa com 10 mil linhas, todas dentro de uma unica funcao. Como voce encontraria o trecho que calcula impostos? Como saberia quais variaveis pertencem a qual parte da logica? Se um calculo aparece em tres lugares diferentes e voce precisa corrigi-lo, como garantiria que corrigiu todos? Funcoes sao a resposta para todos esses problemas. Elas sao o mecanismo fundamental de **abstracao** em programacao: voce da um nome a um bloco de codigo, declara quais dados ele precisa receber e qual resultado produz, e a partir dai pode usa-lo em qualquer lugar como se fosse uma operacao primitiva da linguagem.

Pense em funcoes como receitas culinarias. Uma receita de bolo tem um nome ('Bolo de Chocolate'), uma lista de ingredientes necessarios (farinha, ovos, acucar, chocolate) e um procedimento que transforma esses ingredientes em um resultado (o bolo). Em SpectraLang, uma funcao tem um nome, uma lista de **parametros** (cada um com nome e tipo), um **tipo de retorno** (o que a funcao produz), e um **corpo** (o bloco de codigo que realiza a transformacao). Assim como voce pode usar a mesma receita para fazer quantos bolos quiser, pode chamar a mesma funcao com diferentes argumentos para processar diferentes dados. A palavra-chave e `fn`.

SpectraLang oferece uma conveniencia expressiva chamada **retorno implicito**: a ultima expressao do corpo (aquela que nao termina com ponto e virgula) e automaticamente retornada, sem necessidade da palavra-chave `return`. Isso encoraja um estilo onde funcoes sao compostas de expressoes que fluem naturalmente para um resultado. Quando voce precisa sair da funcao antes do final -- por exemplo, ao detectar uma condicao de erro logo no inicio -- o `return` explicito continua disponivel e e a ferramenta correta. Essa dualidade -- retorno implicito para o caminho feliz, retorno explicito para saidas antecipadas -- combina expressividade com controle.


```
1 // Funcao basica: parametros com tipo, retorno declarado
2   fn soma(a: int, b: int) -> int {
3     return a + b;
4   }
5
6   // Sem retorno declarado: retorna unit (equivalente a void)
7   fn saudacao() {
8     println("Ola!");
9   }
10
11   // Retorno implicito: ultima expressao sem ponto-e-virgula
12   fn eh_par(n: int) -> bool { n % 2 == 0 }
13   fn dobrar(x: int) -> int { x * 2 }
14
15   // Retorno antecipado com return explicito
16   fn dividir_seguro(a: int, b: int) -> int {
17     if b == 0 { return 0; }
18     a / b
19   }
20
21   // if como expressao dentro de funcao
22   fn maximo(a: int, b: int) -> int {
23     if a > b { a } else { b }
24   }
```

Funcoes com retorno explicito, implicito e antecipado.

Visibilidade e Funcoes como Valores

Por padrao, toda funcao em SpectraLang e **privada** ao modulo onde e definida. Para exporta-la, prefixe-a com `pub`. A funcao `main` e sempre declarada como `pub` porque o runtime precisa encontra-la. Este sistema de visibilidade se aplica a todos os itens de nivel superior: structs, enums, traits, constantes e re-exports.

Em SpectraLang, funcoes podem ser tratadas como valores: voce pode passar uma funcao como argumento para outra, armazena-la em uma variavel, ou retorna-la como resultado. O tipo de uma funcao e escrito como `fn(Parametros) -> Retorno`. Alem das funcoes nomeadas, SpectraLang suporta **closures** (lambdas): funcoes anonimas definidas inline com a sintaxe `|params| corpo`. Closures podem capturar variaveis do escopo onde sao definidas -- uma capacidade que funcoes nomeadas nao tem. Exploraremos closures em detalhe no Capitulo 15.

```
1 // Funcao que recebe outra funcao como argumento
2   fn aplicar(x: int, f: fn(int) -> int) -> int {
3     f(x)
4   }
5
6   // Passando uma closure inline
7   let resultado = aplicar(5, |x: int| x * x); // 25
8
9   // Closure armazenada em variavel
10  let dobro = |x: int| x * 2;
11  let oito = dobro(4); // 8
```

PARTE - CAPITULO 9

Arrays, Tuplas e Intervalos

Capitulo 9 -- Arrays, Tuplas e Intervalos

Variaveis escalares -- um numero, uma string, um booleano -- resolvem muitos problemas, mas nao todos. Quando voce precisa trabalhar com uma lista de notas de alunos, uma tabela de precos de produtos, as coordenadas x-y de um ponto no plano, ou as dimensoes de uma matriz para algebra linear, variaveis individuais se tornam inadequadas. Precisamos de tipos que agrupem multiplos valores. SpectraLang oferece arrays e tuplas para esse fim, cada um com caracteristicas distintas.

Arrays sao colecoes *homogeneas*: todos os elementos devem ser do mesmo tipo. Voce cria um array com a sintaxe de colchetes: `[1, 2, 3, 4, 5]`. O acesso aos elementos e por indice baseado em zero: `arr[0]` e o primeiro elemento. Voce pode modificar elementos individuais: `arr[2] = 99`; Arrays tem tamanho fixo apos a criacao -- se voce precisa de uma colecao que cresce dinamicamente, use as listas do modulo `std.collections`. Arrays suportam multiplas dimensoes: `[[1,2],[3,4]]` e uma matriz 2x2.

Tuplas sao colecoes *heterogeneas* de tamanho fixo: cada posicao pode ter um tipo diferente. Voce cria uma tupla com parenteses: `(42, "resposta")`. O acesso e por indice com notacao de ponto: `tupla.0`, `tupla.1`. Tuplas sao particularmente uteis quando uma funcao precisa retornar multiplos valores -- em vez de criar uma struct para um retorno unico, voce pode retornar uma tupla, e o chamador pode desestruturar-la com `let (a, b) = funcao()`; **Intervalos** (ranges) completam a familia de colecoes. Criados com `0..5` (exclusivo) ou `1..=5` (inclusivo), sao usados principalmente com loops `for`, mas tambem podem ser armazenados em variaveis e passados para funcoes.

```
1 // Arrays: elementos do mesmo tipo, indexados por [indice]
2   let nums = [1, 2, 3, 4, 5];
3   let nomes = ["Alice", "Bob", "Carol"];
4   let primeiro = nums[0];    // 1
5   nums[4] = 99;              // modificacao indexada
6
7   // Iteracao com for
8   for i in 0..5 {
9     println(f"nums[{i}] = {nums[i]}");
10  }
11
12  // Arrays multidimensionais
13  let matriz = [[1,2,3], [4,5,6], [7,8,9]];
14  let elem = matriz[1][2];    // 6
15
16  // Tuplas: tipos heterogeneos, acesso por .0, .1, .2...
17  let par = (42, "resposta");
18  let n = par.0;              // 42
19  let s = par.1;              // "resposta"
20
21  // Desestruturacao de tupla
22  let (a, b) = (10, 20);
23
24  // Intervalos (ranges)
25  let r1 = 0..5;              // exclusivo: 0,1,2,3,4
26  let r2 = 1..=5;             // inclusivo: 1,2,3,4,5
```

Arrays, tuplas e intervalos. Cada um resolve um problema diferente de agrupamento.

PARTE - CAPÍTULO 10

Structs e Metodos

Capitulo 10 -- Structs e Metodos

Arrays e tuplas agrupam valores, mas tem uma limitacao: os elementos sao identificados por posicao, nao por nome. Numa tupla ("Alice", 30, 1.70), voce precisa lembrar que .0 e o nome, .1 e a idade e .2 e a altura. Num programa pequeno isso funciona; num sistema com dezenas de tipos de dados diferentes, e uma receita para confusao. **Structs** resolvem esse problema permitindo que voce defina tipos com **campos nomeados**. Cada campo tem um nome e um tipo, e o compilador verifica que voce esta acessando campos que realmente existem.

Uma struct e declarada com a palavra-chave `struct`, seguida pelo nome do tipo (PascalCase), e um bloco com as declaracoes dos campos. Para criar uma instancia, use a sintaxe `NomeDaStruct { campo: valor, ... }`. SpectraLang suporta **field shorthand**: se uma variavel local tem o mesmo nome que um campo, voce pode omitir o `campo:` e escrever apenas o nome. Structs podem ser aninhadas: um campo de uma struct pode ser outra struct, com acesso encadeado via `ponto`.

Alem de armazenar dados, structs podem ter **metodos**: funcoes associadas ao tipo, declaradas em blocos `impl NomeDaStruct { ... }`. Metodos recebem um parametro especial chamado **receptor**, que pode ser `self` (toma posse do valor), `&self` (referencia imutavel), ou `&mut; self` (referencia mutavel). Metodos que nao recebem `self` sao chamados de **metodos estaticos** e sao chamados com `NomeDaStruct::metodo()`. Metodos de instancia sao chamados com notacao de ponto: `valor.metodo()`.

O design de metodos suporta **encadeamento** (method chaining): se um metodo retorna um valor do mesmo tipo, voce pode chamar outro metodo sobre o resultado, formando pipelines expressivos como `ponto.mover(1,2).escalar(3).para_string()`. Cada chamada produz um novo valor, que alimenta a proxima chamada. Esse estilo resulta em codigo que descreve transformacoes de dados como uma sequencia de passos nomeados.

```
1 // Declaracao de struct
2 struct Pessoa {
3     nome: string,
4     idade: int,
5     altura: float,
6     ativo: bool
7 }
8
9 // Instanciacao -- note o field shorthand
10 let nome = "Alice";
11 let idade = 30;
12 let p = Pessoa { nome, idade, altura: 1.72, ativo: true };
13
14 // Struct aninhada
15 struct Endereco { rua: string, numero: int, cidade: string }
16 struct Cliente { nome: string, endereco: Endereco }
17 let c = Cliente {
18     nome: "Bob",
19     endereco: Endereco { rua: "Av. Principal", numero: 100, cidade: "Sao Paulo" }
20 };
21 let cidade = c.endereco.cidade; // acesso aninhado
```

Declaracao, instanciacao e acesso a campos de structs.

```
1 // Bloco impl com metodos staticos e de instancia
2 impl Pessoa {
3     // Metodo estatico (construtor): nao recebe self
4     fn nova(nome: string, idade: int) -> Pessoa {
5         Pessoa { nome, idade, altura: 0.0, ativo: true }
6     }
7
8     // Metodo de instancia com &self (referencia imutavel)
9     fn descrever(&self) -> string {
10         f"{self.nome}, {self.idade} anos"
11     }
12
13     // Metodo que retorna nova instancia modificada
14     fn envelhecer(&self, anos: int) -> Pessoa {
15         Pessoa { idade: self.idade + anos, ..self }
16     }
17
18     // Metodo com &mut self (referencia mutavel) -- modifica in-place
19     fn ativar(&mut self) {
20         self.ativo = true;
21     }
22 }
23
24 // Uso com encadeamento
25 let alice = Pessoa::nova("Alice", 30);
26 let mais_velha = alice.envelhecer(10).envelhecer(5); // method chaining
27 println(alice.descrever());
```

Metodos: staticos, de instancia (&self), mutaveis (&mut self), encadeamento.

PARTE - CAPITULO 11

Enums e Pattern Matching

Capitulo 11 -- Enums e Pattern Matching

Structs modelam a conjuncao 'E': uma Pessoa tem um nome **E** uma idade **E** uma altura. Mas muitos dominios envolvem disjuncao 'OU': um resultado pode ser sucesso **OU** falha; uma forma geometrica pode ser um circulo **OU** um retangulo. Modelar essas situacoes com structs e possivel mas fragil -- voce acabaria com flags booleanas e campos opcionais, e o compilador nao poderia ajudar a garantir que voce tratou todos os casos. **Enums** (tipos algebricos de soma) sao a ferramenta correta para modelar disjuncoes de forma segura e explicita.

Um enum declara um conjunto finito de **variantes**, cada uma das quais pode opcionalmente carregar dados. A variante mais simples e a **unitaria** -- apenas um nome, sem dados associados. `enum Cor { Vermelho, Verde, Azul }` define tres valores possiveis. Variantes tambem podem carregar dados em forma de tupla (`Mover(int, int)`) ou struct (`Circulo { raio: float }`). Para usar o valor dentro de uma variante, voce precisa fazer **pattern matching**: o compilador forca voce a considerar cada variante possivel, garantindo que nenhum caso seja esquecido.

A construcao principal de pattern matching e `match`. O compilador realiza duas verificacoes cruciais: **exaustividade** (todo valor possivel deve ser coberto por pelo menos um padrao) e **utilidade de padroes** (se um padrao nunca pode ser alcancado, um aviso e emitido). SpectraLang oferece padroes literais, de identificador, de enum, OR-patterns, e o curinga `_`. Duas construcoes complementares simplificam casos comuns: `if let padrao = expressao { ... }` executa o bloco apenas se o padrao casar, e `while let padrao = expressao { ... }` combina loop com matching -- util para processar itens de uma fila ate que ela se esgote.

```
1 // Enums com variantes unitarias, de tupla e de struct
2 enum Cor { Vermelho, Verde, Azul }
3 enum Mensagem { Sair, Mover(int, int), Texto(string) }
4 enum Forma {
5     Circulo { raio: float },
6     Retangulo { largura: float, altura: float },
7     Ponto
8 }
9
10 // match com padroes literais
11 let x = 5;
12 let resultado = match x {
13     1 => "um", 2 => "dois", 3 => "tres",
14     4 => "quatro", 5 => "cinco",
15     _ => "outro"
16 };
17
18 // match com padroes de enum
19 match msg {
20     Mensagem::Sair => println("Saindo"),
21     Mensagem::Mover(x, y) => println(f"Mover para ({x},{y})"),
22     Mensagem::Texto(t) => println(t)
23 }
24
25 // OR-pattern
26 enum Token { Plus, Minus, Number(int) }
27 fn peso(token: Token) -> int {
28     match token {
29         Token::Number(value) => value,
30         Token::Plus | Token::Minus => 1
31     }
32 }
33
34 // if let e while let
35 if let Opcao::Algum(valor) = talvez {
36     println(f"Tenho um valor: {valor}");
37 }
38 while let Opcao::Algum(item) = fila.pop() {
39     processar(item);
40 }
41
42 // Metodos em enums
43 impl Forma {
44     fn area(&self) -> float {
45         match self {
46             Forma::Circulo { raio } => 3.14159 * raio * raio,
47             Forma::Retangulo { largura, altura } => largura * altura,
48             Forma::Ponto => 0.0
49         }
50     }
51 }
```

Pattern matching com match, if let, while let. Enums com metodos.

PARTE - CAPITULO 12

Generics --Codigo Que Funciona com Qualquer Tipo

Capitulo 12 -- Generics --Codigo Que Funciona com Qualquer Tipo

Suponha que voce precise escrever uma funcao que retorna o primeiro elemento de um array. Para arrays de inteiros, voce escreveria `fn primeiro_int(arr: [int]) -> int { arr[0] }`. Para arrays de strings, outra funcao identica. Para arrays de booleanos, mais uma. A logica e sempre a mesma -- 'retorne o elemento na posicao zero' -- mas o sistema de tipos exige uma funcao separada para cada tipo de elemento. Isso viola um principio fundamental da engenharia de software: **DRY (Don't Repeat Yourself)**. Para cada nova versao da mesma logica, voce introduz uma oportunidade de erro e um ponto de manutencao adicional.

Generics resolvem esse problema permitindo que voce escreva codigo parametrizado por tipos. Em vez de comprometer-se com um tipo especifico, voce declara um **parametro de tipo** -- por convencao, uma letra maiuscula curta como `T` -- e escreve o codigo em termos desse parametro. O compilador entao **monomorfiza** o codigo: para cada combinacao concreta de tipos com que a funcao generica e usada, ele gera uma versao especializada. Se voce chamar `primeiro<int>` e `primeiro<string>`, duas versoes distintas sao geradas, cada uma otimizada para seu tipo concreto. O resultado e codigo generico no fonte e codigo especializado no binario -- o melhor dos dois mundos.

Generics nao se limitam a funcoes. Structs, enums, traits e blocos `impl` podem ser parametrizados. Uma struct generica como `struct Par<T> { primeiro: T, segundo: T }` pode ser instanciada com qualquer tipo. Isso permite criar estruturas de dados reutilizaveis -- pilhas, filas, arvores -- que funcionam com qualquer tipo de elemento. A biblioteca padrao de SpectralLang faz uso extensivo de generics: `Option<T>`, `Result<T, E>`, e as listas de `std.collections` sao todos generics.

As vezes, porem, codigo generico puro e generico demais. Se voce escrever `fn ordenar<T>(arr: [T]) { ... }`, o compilador nao tem como saber se `T` suporta comparacao. Para resolver isso, SpectralLang permite **trait bounds**: voce restringe o parametro de tipo a tipos que implementam um determinado trait. `fn ordenar<T: Comparavel>(arr: [T])` diz: 'T pode ser qualquer tipo, desde que implemente Comparavel'. Com essa restricao, o compilador sabe que `<` e valido para `T`. Multiplos bounds sao separados por `+`: `T: Exibivel + Comparavel`.


```
1 // Funcao generica -- T e um parametro de tipo
2   fn primeiro<T>(arr: [T]) -> T { arr[0] }
3   fn trocar<T>(a: T, b: T) -> (T, T) { (b, a) }
4
5   // Chamadas: o tipo e inferido pelo argumento
6   let i = primeiro([1, 2, 3]); // T = int
7   let s = primeiro(["a", "b", "c"]); // T = string
8
9   // Struct generica
10  struct Pilha<T> {
11      dados: [T],
12      tamanho: int
13  }
14
15  impl Pilha<T> {
16      fn nova() -> Pilha<T> {
17          Pilha { dados: [], tamanho: 0 }
18      }
19      fn topo(&self) -> T {
20          self.dados[self.tamanho - 1]
21      }
22  }
23
24  // Com trait bound -- restringe T a tipos comparaveis
25  fn ordenar<T: Comparavel>(arr: [T], n: int) { /* ... */ }
26  fn exibir_e_comparar<T: Exibivel + Comparavel>(a: T, b: T) { /* ... */ }
```

Funcoes e structs genericas. Trait bounds restringem os tipos aceitos.

PARTE - CAPITULO 13

Traits -- Definindo Comportamento Compartilhado

Capitulo 13 -- Traits -- Definindo Comportamento Compartilhado

Generics resolvem o problema de escrever código que funciona com múltiplos tipos. Mas há um segundo problema, igualmente importante: como garantir que tipos diferentes possam ser usados de forma intercambiável quando compartilham um comportamento comum? Suponha que você queira escrever uma função `desenhar()` que funcione com círculos, retângulos e triângulos -- formas diferentes, mas que todas podem ser desenhadas. Se você fizer três funções separadas, perde a capacidade de tratar todas as formas uniformemente. A resposta são **traits**.

Um trait é uma declaração de contrato. Ele especifica um conjunto de assinaturas de funções que um tipo deve implementar para satisfazer o trait. A declaração usa a palavra-chave `trait`: `trait Desenhavel { fn desenhar(&self;) -> string; }`. Isso diz: 'qualquer tipo que afirme ser `Desenhavel` deve fornecer uma função `desenhar`'. Um tipo concreto implementa o trait com um bloco `impl Trait for Tipo { ... }`. A partir desse momento, o tipo satisfaz o contrato, e qualquer código que espere um `Desenhavel` pode receber o tipo.

Traits podem incluir **implementações padrão**: métodos cujo corpo é fornecido na própria declaração do trait, e que os implementadores podem opcionalmente sobrescrever. Traits também suportam **herança**: um trait pode exigir que o tipo implemente outro trait como pré-requisito. `trait Animado: Exibivel { ... }` significa: para implementar `Animado`, o tipo precisa primeiro implementar `Exibivel`. Isso cria hierarquias de contratos onde traits mais específicos estendem traits mais gerais. O sistema de traits de SpectraLang é inspirado no de Rust, mas simplificado. O foco está em permitir polimorfismo paramétrico via generics com trait bounds como o mecanismo principal de abstração.

```
1 // Declaracao de trait
2 trait Exibivel {
3     fn exibir(&self) -> string;
4 }
5
6 // Trait com implementacao padrao
7 trait Saudavel {
8     fn saudar(&self) -> string;
9     fn saudar_alto(&self) -> string {
10         f"OLA! {self.saudar()}" // usa saudar() e converte
11     }
12 }
13
14 // Heranca de trait: Animado exige Exibivel como pre-requisito
15 trait Animado: Exibivel {
16     fn mover(&self) -> string;
17 }
18
19 // Implementando trait para um tipo concreto
20 impl Exibivel for Pessoa {
21     fn exibir(&self) -> string {
22         f"{self.nome} (idade: {self.idade})"
23     }
24 }
25
26 // Multiplos trait bounds com +
27 fn processar<T: Exibivel + Saudavel>(item: T) -> string {
28     f"{item.exibir()} diz: {item.saudar()}"
29 }
```

Traits definem contratos. Tipos implementam traits. Generics com bounds usam traits como restricoes.

PARTE - CAPITULO 14

Closures e Funcoes de Alta Ordem

Capitulo 14 -- Closures e Funcoes de Alta Ordem

No Capitulo 8, vimos que funcoes podem ser passadas como argumentos e retornadas como valores. Agora vamos explorar essa ideia em profundidade, comecando pelas **closures** -- funcoes anonimas que podem ser definidas inline, no meio de uma expressao, e que tem a capacidade especial de **capturar variaveis** do escopo onde sao criadas.

A sintaxe de uma closure e compacta: barras verticais delimitam os parametros, seguidos pelo corpo. `|x: int| x * 2` define uma closure que recebe um inteiro e retorna seu dobro. Se o corpo for uma unica expressao, nao sao necessarias chaves. Se for um bloco com multiplas instrucoes, use chaves. O tipo de uma closure e `fn(T) -> R` -- o mesmo tipo de uma funcao nomeada, o que significa que closures podem ser usadas em qualquer lugar que aceite um `fn(P) -> R`.

O que distingue closures de funcoes nomeadas e a **captura de ambiente**. Uma closure definida dentro de uma funcao pode referenciar variaveis do escopo externo. Por exemplo: `let delta = 10; let ajustar = |x: int| x + delta;` A closure armazena o valor de `delta` no momento em que e criada e o utiliza toda vez que e chamada. Em SpectraLang, a captura e **por valor** (a closure recebe uma copia do valor). Isso significa que modificar a variavel original depois de criar a closure nao afeta o comportamento da closure.

Este padrao -- criar uma closure que captura dados e retorna-la -- e extremamente poderoso. Ele permite construir comportamento parametrizado sem criar structs explicitas. Considere: `fn criar_multiplicador(fator: int) -> fn(int) -> int { |x: int| x * fator }`. Esta funcao recebe um fator e retorna uma closure que multiplica qualquer numero por esse fator. Voce pode criar `let dobrar = criar_multiplicador(2);` e `let triplicar = criar_multiplicador(3);` -- duas funcoes diferentes, cada uma com seu proprio fator capturado:

```
1 // Closures basicas
2   let dobro = |x: int| x * 2;
3   let soma = |a: int, b: int| a + b;
4   let quarenta_e_dois = || 42;
5
6   // Closure com bloco
7   let valor_abs = |x: int| {
8     if x < 0 { x * -1 } else { x }
9   };
10
11  // Passando closure como argumento
12  fn aplicar(x: int, f: fn(int) -> int) -> int { f(x) }
13  let r = aplicar(5, |n: int| n * n); // 25
14
15  // Retornando closure que captura o ambiente
16  fn criar_multiplicador(fator: int) -> fn(int) -> int {
17    |x: int| x * fator // 'fator' e capturado por valor
18  }
19
20  let triplicar = criar_multiplicador(3);
21  let resultado = triplicar(10); // 30
```

Closures capturam variaveis do escopo externo por valor.

PARTE - CAPITULO 15

Option, Result, e o Operador ?

Capitulo 15 -- Option, Result, e o Operador ?

Dois problemas são tão universais em programação que merecem tipos dedicados na própria linguagem. O primeiro: uma operação que pode ou não produzir um valor -- como buscar um elemento num array pelo índice, onde o índice pode estar fora dos limites. O segundo: uma operação que pode ter sucesso ou falhar -- como abrir um arquivo que pode não existir, ou analisar uma string como número quando ela pode não conter um número válido. Em muitas linguagens, esses problemas são resolvidos com valores sentinela (null, -1) ou exceções (try/catch). SpectraLang adota uma abordagem diferente e mais segura: representa essas situações como **tipos** que o compilador força você a tratar explicitamente.

`Option<T>` resolve o primeiro problema. É um enum com duas variantes: `Option::Some(valor)` significa 'aqui está o valor', e `Option::None` significa 'não há valor'. Quando uma função retorna `Option<int>`, você não pode simplesmente usar o resultado como se fosse um inteiro -- o compilador exige que você verifique se é `Some` ou `None` antes de acessar o valor. Isso elimina o infame 'null pointer exception' da classe de erros possíveis: em SpectraLang, você não pode esquecer de tratar a ausência de valor porque o sistema de tipos não deixa.

`Result<T, E>` generaliza `Option` para casos onde o que interessa não é apenas a ausência de valor, mas o *motivo* da falha. Tem duas variantes: `Result::Ok(valor)` para sucesso e `Result::Err(erro)` para falha. O tipo `E` pode ser qualquer coisa -- uma string com a mensagem de erro, um enum que descreve diferentes modos de falha, ou até mesmo um inteiro com um código de erro. Isso permite que funções comuniquem não apenas que falharam, mas por que falharam.

O operador `?` é o coroamento do sistema de tratamento de erros. Ele desembulha um `Result` ou `Option`: se for `Ok/Some`, extrai o valor e continua; se for `Err/None`, retorna antecipadamente da função com o erro. Isso transforma código que seria uma cascata de `match` aninhados em uma sequência linear elegante. A função que usa `?` deve ter retorno compatível com o tipo propagado. O módulo `std.option` e `std.result` fornecem funções auxiliares como `is_some`, `unwrap`, e `unwrap_or` para casos onde um `match` seria excessivamente verboso.

```
1 // Option<T>; valor que pode ou nao existir
2 fn dividir_seguro(a: int, b: int) -> Option<int> {
3     if b == 0 { return Option::None; }
4     Option::Some(a / b)
5 }
6
7 match dividir_seguro(10, 2) {
8     Option::Some(v) => println(f"Resultado: {v}"),
9     Option::None    => println("Divisao por zero!")
10 }
11
12 // Result<T, E>; sucesso ou falha com motivo
13 fn dividir(a: int, b: int) -> Result<int, string> {
14     if b == 0 { return Result::Err("divisao por zero"); }
15     Result::Ok(a / b)
16 }
17
18 match dividir(10, 2) {
19     Result::Ok(n) => println(f"Sucesso: {n}"),
20     Result::Err(msg) => println(f"Erro: {msg}")
21 }
22
23 // Operador ? -- desembrulha ou retorna erro
24 fn processar(entrada: string) -> Result<int, string> {
25     let n = analisar_inteiro(entrada)?;
26     Result::Ok(n * 2)
27 }
28
29 // Encadeamento elegante com ?
30 fn pipeline(a: string, b: string) -> Result<int, string> {
31     let x = analisar_inteiro(a)?;
32     let y = analisar_inteiro(b)?;
33     let r = dividir(x, y)?;
34     Result::Ok(r)
35 }
36
37 // Stdlib helpers
38 import std.option as opt;
39 let tem = opt.is_some(Option::Some(42));           // true
40 let val = opt.option_unwrap(Option::Some(42));     // 42
41 let def = opt.option_unwrap_or(Option::None, 0);   // 0
42
43 import std.result as res;
44 let ok = res.is_ok(Result::Ok(100));               // true
45 let v = res.result_unwrap(Result::Ok(42));         // 42
46 let d = res.result_unwrap_or(Result::Err("e"), 0); // 0
```

Option para valores opcionais. Result para operacoes falháveis. ? para propagacao.

PARTE - CAPITULO 16

Sistema de Modulos e Visibilidade

Capitulo 16 -- Sistema de Modulos e Visibilidade

Todo programa SpectraLang que escrevemos ate agora comecou com uma declaracao module seguida por importacoes. E hora de entender exatamente como o sistema de modulos funciona. Modulos sao a unidade fundamental de organizacao de codigo em SpectraLang: cada arquivo e um modulo, e modulos podem ser aninhados em uma hierarquia que espelha a estrutura de diretorios. O sistema de modulos serve a tres propositos: organizar codigo em unidades logicas, controlar a visibilidade (o que e publico e o que e privado), e gerenciar dependencias entre partes do programa.

A importacao oferece quatro formas. A **qualificada** (`import std.io;`) importa o modulo e requer que voce use o caminho completo para acessar seus itens (`std.io.println(...)`). Com **alias** (`import std.math as math;`) voce encurta o prefixo. A importacao **por nome** (`import { println } from std.io;`) traz itens especificos para o escopo local. E o **re-export** (`pub import { ... } from ...;`) torna itens importados disponiveis para quem importar o seu modulo. A escolha entre qualificada e por nome e questao de estilo: importacao qualificada evita conflitos de nome e deixa claro de onde cada simbolo vem; importacao por nome reduz verbosidade.

SpectraLang tem tres niveis de visibilidade. O padrao (sem modificador) e **privado**: apenas o modulo atual pode acessar. `pub` torna o item **publico**, acessivel de qualquer modulo. `internal` restringe o acesso ao **pacote** -- util para expor APIs entre modulos do mesmo projeto sem torna-las parte da interface publica permanente. O compilador impoe regras de consistencia: funcoes `pub` nao podem expor tipos privados em suas assinaturas, structs `pub` nao podem ter campos de tipos privados, e assim por diante. Essas regras garantem que o encapsulamento nao possa ser violado acidentalmente.

```
1 // Formas de importacao
2 import std.io;                // qualificada
3 import std.math as math;      // alias
4 import { println } from std.io; // por nome
5 pub import { println } from std.io; // re-export
6
7 // Visibilidade de 3 niveis
8 module minha.biblioteca;
9 pub struct Ponto { pub x: int, pub y: int } // campos publicos
10 internal fn util() -&gt; int { 42 }           // pacote
11 fn helper() -&gt; int { util() }              // privada
```


A biblioteca padrao e organizada em modulos funcionais. `std.io` para entrada e saida, `std.string` para manipulacao de texto, `std.math` para funcoes matematicas, `std.convert` para conversao de tipos, `std.collections` para listas dinamicas, `std.random` para numeros aleatorios, `std.fs` para sistema de arquivos, `std.env` para variaveis de ambiente, `std.time` para tempo e duracao, `std.tensor` para tensores, `std.ml` para machine learning, `std.concurrent` para concorrencia, e `std.serve` para serving e guardrails. Cada um desses modulos sera explorado nas Partes VII a IX.

PARTE - CAPITULO 17

std.io, std.string, std.char, std.math, std.convert

Capitulo 17 -- std.io, std.string, std.char, std.math, std.convert

std.io -- Entrada e Saida

```
1 import std.io;
2
3     println("Ola, mundo!");    // imprime com nova linha
4     print("Sem ");             // sem nova linha
5     print("nova linha");
6     println("!");              // "Sem nova linha!"
7
8     eprint("Erro: ");           // stderr sem nova linha
9     eprintln("arquivo nao encontrado"); // stderr com nova linha
10
11     flush();                   // esvazia buffer de saida
12     let nome = input("Digite seu nome: "); // prompt + leitura
13     let linha = read_line();    // le uma linha da entrada padrao
```

std.string -- Manipulacao de Strings

O modulo `std.string` oferece funcoes para inspecao, transformacao e extracao de strings. As funcoes operam sobre indices de bytes na codificacao UTF-8 -- isso importa quando voce trabalha com caracteres nao-ASCII que ocupam multiplos bytes.

Funcao	Assinatura	Descricao
<code>len</code>	<code>(string) -> int</code>	Numero de bytes
<code>contains</code>	<code>(string, string) -> bool</code>	Contem substring?
<code>to_upper</code>	<code>(string) -> string</code>	Converte ASCII para maiusculo
<code>to_lower</code>	<code>(string) -> string</code>	Converte ASCII para minusculo
<code>trim</code>	<code>(string) -> string</code>	Remove whitespace das pontas
<code>starts_with</code>	<code>(s, prefix) -> bool</code>	Comeca com prefixo?
<code>ends_with</code>	<code>(s, suffix) -> bool</code>	Termina com sufixo?
<code>concat</code>	<code>(a, b) -> string</code>	Concatena duas strings
<code>repeat_str</code>	<code>(s, n) -> string</code>	Repete a string n vezes

Funcao	Assinatura	Descricao
char_at	(s, idx) -> int	Codigo Unicode na posicao; -1 se out-of-bounds
substring	(s, start, end) -> string	Extrai substring [start, end)
replace	(s, from, to) -> string	Substitui todas as ocorrencias
index_of	(s, sub) -> int	Posicao da primeira ocorrencia; -1 se nao encontrada
split_first	(s, sep) -> string	Parte antes do primeiro separador
split_last	(s, sep) -> string	Parte apos o ultimo separador
count_occurrences	(s, sub) -> int	Conta ocorrencias da substring
is_empty	(string) -> bool	String vazia?
pad_left	(s, w, ch) -> string	Preenche a esquerda com char
pad_right	(s, w, ch) -> string	Preenche a direita com char
reverse_str	(string) -> string	Inverte a string

std.char -- Operacoes em Caracteres

As funcoes de `std.char` operam sobre codigos Unicode (inteiros). Para usar com caracteres de uma string, obtenha o codigo via `std.string.char_at()` primeiro.

```

1  import std.char;
2      is_alpha(c: int) -> bool      // e letra?
3      is_digit_char(c: int) -> bool // e digito?
4      is_whitespace_char(c: int) -> bool // e whitespace?
5      is_upper_char(c: int) -> bool // e maiuscula?
6      is_lower_char(c: int) -> bool // e minuscula?
7      is_alphanumeric(c: int) -> bool // e alfanumerico?
8      to_upper_char(c: int) -> int  // converte para maiuscula
9      to_lower_char(c: int) -> int  // converte para minuscula

```

Exemplo: `std.char.is_alpha(65) -> true ('A')`; `std.char.is_digit_char(48) -> true ('0')`.

std.math -- Funcoes Matematicas

Categoria	Funcoes
Inteiros	abs, min, max, clamp, sign, gcd, lcm
Floats	abs_f, sqrt_f, pow_f, floor_f, ceil_f, round_f
Trigonometricas	sin_f, cos_f, tan_f, atan2_f (radianos)
Logaritmicas	log_f (ln), log2_f, log10_f
Especiais	is_nan_f, is_infinite_f
Constantes	pi(), e_const()

```
1 import std.math as m;
2     let pi = m.pi();           // ~3.14159
3     let area = pi * m.pow_f(5.0, 2.0); // pi * r^2
4     let hip = m.sqrt_f(9.0 + 16.0); // sqrt(25) = 5.0
5     let c = m.clamp(150, 0, 100); // 100
6     let g = m.gcd(12, 8);       // 4
```

Argumentos trigonometricos em radianos.

std.convert -- Conversao de Tipos

```
1 import std.convert;
2
3 // Para string
4 int_to_string(42)      -> "42"
5 float_to_string(3.14)  -> "3.14"
6 bool_to_string(true)   -> "true"
7
8 // De string
9 string_to_int("123")    -> 123
10 string_to_float("3.14") -> 3.14
11 string_to_int_or("abc", -1) -> -1 // com valor padrao
12
13 // Numericas
14 int_to_float(7)         -> 7.0
15 float_to_int(9.9)       -> 9      (trunca, nao arredonda!)
16 bool_to_int(true)       -> 1
17 string_to_bool("true")  -> true (case-insensitive)
```

PARTE - CAPITULO 18

std.collections, std.random, std.fs, std.env, std.time

Capitulo 18 -- std.collections, std.random, std.fs, std.env, std.time

std.collections -- Listas Dinamicas

Listas sao gerenciadas via handles opacos (inteiros). Nao manipule handles diretamente -- use as funcoes do modulo.

```
1  import std.collections as col;
2      let lista = col.list_new();           // cria lista vazia
3
4      // Mutacao
5      col.list_push(lista, 10);            // adiciona ao final
6      col.list_push(lista, 20);
7      col.list_insert_at(lista, 1, 15);    // insere na posicao 1
8
9      // Acesso
10     let n = col.list_len(lista);          // 3
11     let v = col.list_get(lista, 0);       // 10
12     col.list_set(lista, 0, 99);          // modifica indice 0
13
14     // Busca / ordenacao
15     let tem = col.list_contains(lista, 20); // bool
16     let idx = col.list_index_of(lista, 20); // indice ou -1
17     col.list_sort(lista);                 // ordena in-place
18
19     // Remocao
20     let ultimo = col.list_pop(lista);      // remove e retorna ultimo
21     let primeiro = col.list_pop_front(lista); // remove e retorna primeiro
22
23     // Lifecycle
24     col.list_clear(lista);                 // esvazia
25     col.list_free(lista);                 // libera memoria
26     col.list_free_all();                   // libera todas
```

std.random, std.fs, std.env

```
1  import std.random;
2      random_seed(42);                     // semente para reproducao
3      let dado = random_int(1, 6);         // 1..6 inclusive
4      let f = random_float();              // [0.0, 1.0)
5      let b = random_bool();               // true ou false
```

```

1  import std.fs;
2      let conteudo = fs_read("dados.txt");           // le arquivo -> string
3      let ok = fs_write("saida.txt", "Hello!");      // escreve (cria dirs pais)
4      fs_append("log.txt", "nova linha\n");         // adiciona ao final
5      if fs_exists("config.txt") { /* ... */ }      // verifica existencia
6      fs_remove("temp.txt");                        // remove arquivo

```

Retornos: `fs_read` retorna "" em erro; `fs_write/fs_append` retornam `bool`.

```

1  import std.env;
2      let home = env_get("HOME");                   // obtem variavel de ambiente
3      env_set("MINHA_VAR", "valor");                // define variavel
4      let argc = env_args_count();                  // numero de argumentos CLI
5      for i in 0..argc {
6          println(f"arg[{i}] = {env_arg(i)}");
7      }

```

std.time -- Tempo e Duracao

```

1  import std.time;
2
3      // Wall clock
4      let s = time_now_secs();                       // segundos desde epoch Unix
5      let ms = time_now_millis();                    // milissegundos desde epoch Unix
6
7      // Monotonic clock (nao afetado por ajustes do sistema)
8      let m = monotonic_millis();
9      let ns = monotonic_nanos();
10
11     // Duration
12     let d = duration_ms(500);                       // 500ms
13     let ds = duration_secs(5);                      // 5s
14     let dms = duration_millis(d);                   // extrai ms
15     let d2 = duration_add(d, d);                     // soma com checagem de overflow
16     sleep(d);                                        // sleep com Duration
17
18     // Instant
19     let inicio = instant_now();                      // captura instante
20     let decorrido = instant_elapsed_ms(inicio);
21     let futuro = instant_add(inicio, duration_ms(100));
22     if instant_has_elapsed(futuro) { /* ... */ }
23
24     // UTC
25     let dt = unix_to_utc(time_now_secs());
26     let ano = utc_year(dt);
27     let mes = utc_month(dt);
28
29     // Benchmark simples
30     let t0 = time_now_millis();
31     // ... operacao ...
32     let t1 = time_now_millis();
33     println(f"Duracao: {t1 - t0}ms")

```

PARTE - CAPITULO 19

Async/Await -- Concorrencia de Primeira Classe

Capitulo 19 -- Async/Await -- Concorrencia de Primeira Classe

SpectralLang trata programacao assincrona como um conceito nativo da linguagem, nao como uma biblioteca adicionada depois. Instrucoes como `AsyncSuspend` / `AsyncResume` / `AsyncReady` e o tipo `Task<T>` existem no nivel da representacao intermediaria SSA. O compilador rebaixa `async/await` para uma maquina de estados finita, eliminando a necessidade de callbacks, promises ou futures como conceitos de runtime visiveis ao programador. Quando voce escreve `await`, esta instruindo o compilador a gerar um ponto de suspensao na maquina de estados -- o codigo apos o `await` vira um callback implicitamente, mas voce o escreve como se fosse codigo sequencial normal.

O runtime de SpectralLang usa mio sobre uma fila de prioridade, mapeando para **epoll** (Linux), **IOCP** (Windows) e **kqueue** (macOS), selecionado em tempo de compilacao. Este reactor de plataforma gerencia o escalonamento de tasks assincronas, fornecendo a camada de execucao sobre a qual o `async/await` opera. A seguranca e reforcada em tempo de compilacao: o compilador emite codigos de erro para valores `non-Send` que cruzam pontos de `await`, prevenindo condicoes de corrida e violacoes de seguranca de memoria.

```
1  async fn ready_value() -> int { return 41; }
2
3  async fn add_one() -> int {
4      let value = await ready_value();
5      value + 1
6  }
7
8  async fn from_block() -> int {
9      let task: Task<int> = async {
10         let base = await ready_value();
11         base + 1
12     };
13     await task
14 }
15
16 fn main() -> int {
17     let task = add_one();           // cria task sem bloquear
18     let block_task = from_block();
19     0
20 }
```

Fonte: `tests/validation/121_async_await_lowering.spectra`

Concorrencia com std.concurrent

```
1  import std.concurrent as cc;
2
3      let handle = cc.task_spawn(worker_fn);    // spawn de task
4      let result = cc.task_join(handle);        // espera e obtem resultado
5
6      // Channels
7      let ch = cc.channel_new();
8      cc.channel_send(ch, 42);
9      let val = cc.channel_recv(ch);
10     cc.channel_close(ch);
11
12     // Counter atomico
13     let ctr = cc.counter_new();
14     cc.counter_add(ctr, 1);
15     let v = cc.counter_get(ctr);
16
17     // Pipeline
18     let total = cc.pipeline_sum(inputs);
```


PARTE - CAPITULO 20

Tensors e Autodiff

Capitulo 20 -- Tensors e Autodiff

Este e o diferencial mais forte de SpectraLang: **dtype, rank, dimensoes, layout de memoria e dispositivo** sao parte do tipo e verificados em tempo de compilacao. Diferente de Python, onde um erro de forma de tensor so aparece no meio do treinamento (epoca 3, batch 47), SpectraLang rejeita incompatibilidades de rank, dtype, forma, layout e dispositivo antes de qualquer codigo executar. Codigos de erro E1401-E1406 cobrem cada categoria de incompatibilidade.

Tipos Tensor Estaticos

```
1 // Rank 1 com dimensao explicita
2   let v: Tensor<float, rank1, dim3, row_major, cpu> = [1.0, 2.0, 3.0];
3
4   // Rank 1 com dimensao dinamica
5   let any_len: Tensor<float, rank1, dynamic_dim, row_major, cpu> = v;
6
7   // Rank 2 (matriz)
8   let m: Tensor<float, rank2, dim2, dim2, row_major, cpu> = [[1.0, 2.0], [3.0, 4.0]];
9
10  // Funcao com tipo tensor estatico
11  fn vector_total(values: Tensor<float, rank1, dim4, row_major, cpu>) -> int {
12    return tensor.sum_f(values) as int;
13  }
```

Fonte: tests/validation/102_pattern_tensor_ai_composition_stress.spectra

API de Criacao e Metadados

```
1 import std.tensor as tensor;
2
3   let a = tensor.arange(1, 5, 1); // [1, 2, 3, 4]
4   let b = tensor.full(4, 2); // [2, 2, 2, 2]
5   let c = tensor.add(a, b); // [3, 4, 5, 6]
6
7   // Metadados
8   let n = tensor.len(a); // 4
9   let r = tensor.rank(a); // 1
10  let rows = tensor.rows(m); // 2
11
12  // Acesso indexado
13  let val = tensor.get(a, 0); // 1
14  tensor.set(a, 0, 99); // modifica elemento
15
16  // Reshape
17  let m = tensor.reshape(tensor.arange(1, 7, 1), 2, 3); // 2x3 matrix
```

Autodiff: Blocos diff e backward

`diff { }` é um construct de linguagem que rebaixa para `tensor.backward`. O bloco deve produzir um tensor escalar de loss. Operacoes de metadados dentro de `diff { }` falham com codigo E1406.

```
1  tensor.set_grad_enabled(true);
2
3      let initial: Tensor<float, rank1> = [3.0, 3.0, 3.0];
4      let weights = tensor.requires_grad(initial, true);
5
6      let loss: Tensor<float, rank0> = diff {
7          tensor.sum_t(tensor.mul(weights, weights))
8      };
9
10     let grad: Tensor<float, rank1> = tensor.grad(weights);
11     // grad ~ [6.0, 6.0, 6.0] (d/dx de x^2 = 2x, sobre sum)
```

Autodiff é recurso do compilador. PyTorch detecta erro de forma no epoco 3; SpectraLang antes de executar.

PARTE - CAPITULO 21

std.ml -- Machine Learning

Capitulo 21 -- std.ml -- Machine Learning

std.ml fornece a camada de alto nivel para treinamento em CPU sobre handles de std.tensor. Inclui modulos, camadas, funcoes de perda, otimizadores, datasets, dataloaders, experimentos, treinamento distribuido, exportacao ONNX, e componentes de transformer como atencao e embeddings posicionais.

```
1  import std.tensor as tensor;
2      import std.ml as ml;
3
4      fn train_step(x, target, w, b) -> int {
5          let pred = ml.linear(x, w, b);      // camada densa diferenciavel
6          let loss = ml.mse_loss(pred, target); // MSE loss
7          tensor.backward(loss);              // autodiff reverso
8          ml.sgd_step(w, 0.1);                // atualiza parametro
9          ml.sgd_step(b, 0.1);
10         0
11     }
```

Fonte: [examples/ai/mlp_training_serving.spectra](#)

Funcionalidades de ML

Categoria	Funcoes
Module	module_new, module_add_parameter, module_parameter_count
Layers	linear, conv2d, dropout, max_pool2d
Losses	mse_loss, bce_loss, cross_entropy_loss, nll_loss
Optimizers	sgd_step, sgd_momentum_step, adam_step, adamw_step, exp_lr
Datasets	dataset_from_tensors, dataset_from_csv, dataset_from_jsonl, dataset_from_npy
Dataloaders	dataloader_new, dataloader_batch_count, dataloader_batch_features
Dataframes	dataframe_from_csv, dataframe_rows, dataframe_cols, dataframe_column
Experiments	experiment_start, experiment_finish, experiment_log_metric
ONNX	onnx_export, onnx_import_summary, onnx_validate, onnx_roundtrip
Transformers	embedding_lookup, positional_encoding, attention, kv_cache_*, gelu, swiglu
Tokenizer	tokenizer_wordpiece, tokenizer_encode, tokenizer_decode, text_embed
RAG	rag_chunk_text, rag_build_prompt, rag_evaluate_answer, vector_index_*
Evaluation	metrics_classification, metrics_regression, serving_metrics, evaluation_report

PARTE - CAPITULO 22

spectra.api -- HTTP, Routing, Middleware, e Database

Capitulo 22 -- spectra.api -- HTTP, Routing, Middleware, e Database

O pacote `spectra.api` é a plataforma nativa de API de SpectraLang. Tipos HTTP são structs de primeira classe (`Request`, `Response`, `Method`, `Status`, `Header`, `Cookie`), não mapas ou dicionários genéricos. O contrato define 211 host calls que cobrem HTTP/1.1 completo, JSON com derive macros, WebSocket, SSE, CORS, autenticação, rate limiting, upload de arquivos, e drivers para SQLite, PostgreSQL e Redis. O servidor mínimo cabe em 12 linhas:

Hello HTTP -- Servidor Mínimo

```
1  module api.hello;
2
3      import std.api.http;
4      import std.api.routing;
5      import std.api.handler;
6      import std.api.server;
7      import std.concurrent as cc;
8
9      fn hello_handler(request: Request) -> Response {
10         Response::ok("Hello, Spectra API!", "text/plain")
11     }
12
13     pub fn main() -> int {
14         let router = Router::new();
15         Router::get(router, "/hello", hello_handler);
16
17         let srv = Server::new("127.0.0.1:8080");
18         Server::serve(srv, router, |err: string| {
19             println(f"Server error: {err}");
20         });
21
22         println("Server running on http://127.0.0.1:8080");
23         return 0;
24     }
```

Fonte: `examples/api/00_hello_http.spectra`

Routing, Middleware, e JSON

```

1 // Padroes de rota
2 Router::get(router, "/users", list_users); // literal
3 Router::get(router, "/users/{id}", get_user); // parametro
4 Router::get(router, "/users/{id:\\d+}", get_user_id); // com regex
5 Router::get(router, "/files/{*path}", serve_file); // wildcard
6
7 // Middleware chain
8 fn logging_middleware(req: Request, next: fn(Request) -> Response) -> Response {
9     println(f"{Method::to_string(req.method)} {req.path}");
10    let resp = next(req);
11    println(f"-> {Status::code(resp.status)}");
12    resp
13 }
14
15 let chain = MiddlewareChain::new();
16 MiddlewareChain::add(chain, logging_middleware);
17 MiddlewareChain::add(chain, auth_middleware);
18 Server::serve_with_middleware(srv, router, chain, error_handler);
19
20 // JSON com derive macros
21 #[derive(Serialize, Deserialize)]
22 struct User {
23     id: int,
24     name: string,
25     #[rename("email_address")]
26     email: string,
27     age: Option<int>;
28 }

```

Database Drivers

```

1 // SQLite
2 import std.api.sqlite as db;
3 let conn = SqlConnection::open("app.db");
4 let stmt = SqliteStatement::prepare(conn,
5     "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT NOT NULL)");
6 SqliteStatement::step(stmt);
7 SqliteStatement::finalize(stmt);
8
9 // INSERT com bind parameters
10 let insert = SqliteStatement::prepare(conn,
11     "INSERT INTO users (name, email) VALUES (?, ?)");
12 SqliteStatement::bind_text(insert, 1, "Alice");
13 SqliteStatement::step(insert);
14
15 // SELECT
16 let query = SqliteStatement::prepare(conn,
17     "SELECT id, name FROM users ORDER BY id");
18 while SqliteStatement::step(query) != 0 {
19     let id = SqliteStatement::column_int(query, 0);
20     let name = SqliteStatement::column_text(query, 1);
21     println(f"#{id}: {name}");
22 }
23 SqlConnection::close(conn);

```

Drivers adicionais via `std.api.postgres` (prepared statements, transacoes, COPY, LISTEN/NOTIFY) e `std.api.redis` (get, set, delete, expire, pub/sub). Observabilidade integrada: health checks (/startupz, /readyz, /healthz), Prometheus metrics, e OpenTelemetry tracing via OTLP/HTTP.

PARTE - CAPITULO 23

O Pipeline de Compilacao

Capitulo 23 -- O Pipeline de Compilacao

De um arquivo `.spectra` a execucao, o caminho passa por tres grandes estagios mais a camada de runtime. O binario `spectralang` orquestra tudo via o trait `BackendDriver`, que isola o front-end do back-end -- permitindo que o LSP reutilize o front-end sem carregar Cranelift.



Fluxo: `cli/compiler_integration.rs` -> `CompilationPipeline::compile()` (front-end) -> `FullPipelineBackend::run()/execute()` (mid+back-end) -> runtime.

As Quatro Crates Centrais

O compilador de SpectraLang e organizado em quatro crates Rust. O **spectra-compiler** contem o front-end completo: lexer, parser, definicao da AST, analise semantica multipassada e lint. Zero dependencias de Cranelift ou do runtime -- o que permite que o LSP, o formatador e o linter o reutilizem diretamente. O **spectra-midend** implementa o rebaixamento da AST para IR SSA (SIR), o `TensorGraph` (uma segunda IR de mais alto nivel para operacoes tensoriais), passes de otimizacao como constant folding e dead code elimination, e o no nativo de autodiff. O **spectra-backend** contem o codegen Cranelift tanto para JIT quanto para AOT. E o **spectra-runtime** fornece os servicos de execucao: inicializacao, gerenciamento hibrido de memoria, registro de host calls, e o reactor de eventos para `async/await`.

```
1  main() -&gt; run_cli() -&gt; parse_cli() -&gt; execute_build_command()
2      -&gt; execute_plan_with_sources() -&gt; ProjectPlan::build_with_sources()
3      -&gt; topological_order() -&gt; SPECTRACompiler::compile() (por modulo)
4      -&gt; CompilationPipeline::compile() + FullPipelineBackend::run/execute()
5      -&gt; spectra_runtime::initialize();
6      codegen.execute_entry_point("main", ir)
```

Fonte: `tools/spectra-cli/src/{main,compiler_integration}.rs`

PARTE - CAPITULO 24

Front-end: Lexer, Parser e Semantica

Capitulo 24 -- Front-end: Lexer, Parser e Semantica

Lexer

O lexer é um automato finito escrito a mão sobre `char_indices` da string fonte. Ele emite tokens com spans precisos (offset de byte + linha/coluna), suportando comentários de linha, f-strings, literais de caractere, sequências de escape e operadores compostos (`==`, `<=`, `&&`, `->`, `..=`). Erros lexicos recebem códigos L001-L099 com spans apontando exatamente para o caractere problemático.

Parser

O parser é de descida recursiva com 1 token de lookahead e **recuperação de erro** via `synchronize()`, que reporta múltiplos erros por passada em vez de abortar no primeiro. O `ModuleLoader` implementa cache incremental: hash de *source* + *feature flags* evita re-lexer e re-parser em cache hit. Erros de parser recebem códigos P001-P999.

Análise Semantica Multipassada

A análise semântica usa um padrão visitor sobre a AST em múltiplas passadas. A passada 0 resolve importações via `ModuleRegistry` compartilhado. A passada 1 coleta declarações e detecta duplicatas. A passada 2 analisa corpos de funções, gerencia a pilha de escopos, verifica retornos e valida implementações de `trait`. A passada 3 infere argumentos de tipo genérico. A passada 4 preenche `type_name` em `MethodCall` para o mid-end. Erros semânticos usam códigos E001-E099, com códigos especializados E1401-E1406 para tensores e E2101-E2120 para `async`. Diagnósticos são exportáveis em JSON estavel ou SARIF 2.1.0.

PARTE - CAPITULO 25

Mid-end, Back-end e Runtime

Capitulo 25 -- Mid-end, Back-end e Runtime

IR SSA (SIR)

A representacao intermediaria e baseada em forma SSA: cada funcao e um grafo de BasicBlocks com Instructions e um Terminator. Valores sao identificados por Value { id }, unicos por funcao. Terminadores incluem Return, Branch, CondBranch, Switch, Unreachable. Instrucoes cobrem aritmetica, memoria, chamadas, PHI, constantes tipadas, Cast, e operacoes especificas de linguagem como AutodiffStep (no nativo de autodiff reverso) e AsyncSuspend, AsyncResume, AsyncReady. Genericos sao monomorfizados com name mangling e teto de 512 especializacoes.

Otimizacao

O mid-end implementa passes de otimizacao via o trait Pass { fn name() -> &str; fn run(&mut; self, module: &mut; Module) -> bool; }. ConstantFolding ativo em opt-level >= 1. FunctionInlining + DeadCodeElimination em >= 2. ConcurrentSpawnJoinFusion quando optimize ativo. LoopStructureValidation roda sempre. verify_module() roda antes e depois de cada otimizacao.

Back-end Cranelift e Runtime

O back-end JIT usa JITBuilder com optimizer 'speed'. Cada basic block da IR vira um bloco Cranelift; instrucoes PHI viram block parameters (Cranelift nao tem PHI nativo). O AOT via cranelift_object emite COFF/ELF/Mach-O. O runtime implementa HybridMemory: GC de rastreamento (Gc<T>) para valores gerenciados e alocao manual para scratch de baixo nivel. A ABI de host-call usa SpectraHostValue (int64_t) com contexto de argumentos e resultados. O contrato spectra.api define exatamente 211 host calls assereadas em tempo de compilacao.

PARTE - CAPITULO 26

Toolchain: CLI, Package Manager, LSP

Capitulo 26 -- Toolchain: CLI, Package Manager, LSP

O Binario spectralang

O binario spectralang centraliza todos os comandos da toolchain. compile compila modulos, check verifica tipos sem gerar codigo, run compila e executa via JIT, lint executa regras de lint, fmt formata codigo-fonte, repl inicia o REPL interativo, e new cria um novo projeto. Niveis de otimizacao: -O0 (sem otimizacoes), -O1 (basicas), -O2 (moderadas, padrao), -O3 (agressivas). Diagnosticos exportaveis em JSON ou SARIF.

Package Manager

spectralang package oferece 19 subcomandos (lock, build, test, add, publish, catalog, etc.). O arquivo de lock spectrum.lock e deterministico com SHA-256. O manifesto spectra.toml contem secoes [project], [release] e [dependencies]. Suporte a workspaces multi-crate e resolucao de dependencias transitivas.

LSP e Ferramentas de Desenvolvimento

spectra-lsp implementa 14 capacidades: hover, go-to-definition, rename, completion, inlay hints, semantic tokens, quickfix diagnostics, document symbols, formatting, code actions, folding ranges, signature help, references, e workspace symbols. O LSP reutiliza o front-end do compilador diretamente, sem carregar Cranelift ou o runtime. O formatador usa indentacao de 4 espacos e max 100 caracteres por linha. O linter tem 3 regras: unused-binding, unreachable-code e shadowing. spectralang test executa testes do projeto.

```
1 $ spectralang repl
2   spectra> let x = 42
3   spectra> x * 2
4   84
5   spectra> f"Ola, {x}!"
6   "Ola, 42!"
7   spectra> :quit
```

PARTE - CAPITULO 27

Performance, Maturidade e Roadmap

Capitulo 27 -- Performance, Maturidade e Roadmap

SpectralLang e uma linguagem em desenvolvimento ativo, com foco disciplinado em qualidade de engenharia. O roadmap e rastreado em tres artefatos: o plano estrategico de implementacao, o backlog humano, e o roadmap estruturado em TOML. Aproximadamente 67% dos 267 itens do roadmap estao concluidos, cobrindo as Fases 0-22 (nucleo de IA/ML, async core, API foundation).

Benchmarks Cross-Linguagem

31 cenarios x Spectra/Go/Java/Rust, 3 warmups, 20 amostras, gate de drift $\leq 15\%$:

Cenario	vs Go	vs Rust
tensor-reduce	1.05x (paridade)	1.6x
tensor-elementwise	1.2x	2.0x
tensor-matmul	1.94x	2.5x
cpu-hashmap	6.0x	6.9x
cpu-string-build	71.7x (gap conhecido)	66.9x

Maturidade das Features da Linguagem

Feature	Status
Variables, primitives, operators	Stable
Functions, implicit return, generics	Stable
Control flow (if/unless/while/for/loop/do-while)	Stable
switch, match, if let, while let	Stable
Arrays, tuples, ranges	Stable
Structs, enums, impl methods	Stable
Traits, generic bounds, closures	Stable
Module system, visibility (pub/internal)	Stable
Option, Result, ? operator	Stable
F-strings, implicit block return	Stable
Tensor types (static rank/dtype/dims)	Stable

Feature	Status
Autodiff (diff blocks, backward, grad)	Stable
Async/await, Task<T>	Stable
spectra.api (HTTP, routing, middleware, JSON)	Stable
SQLite/Postgres/Redis drivers	Stable
Otel, Prometheus, health checks	Stable
Package manager, LSP, formatter	Stable
Exact-width numerics (i8..i64, f32, f64)	In progress
Dynamic overflow diagnostics	In progress
GPU backends (CUDA, ROCm, Metal, Vulkan)	Future/Reserved
Distributed training (real multi-node)	Future

Status honesto: ainda nao e v1.0 estavel. JIT e o caminho primario; exe standalone nao esta totalmente integrado; a stdlib tem gaps relativos ao plano. Posicionamento: base de engenharia solida e disciplinada, nao promessa exagerada.

PARTE - CAPITULO 28

Apendice A: Referencia Rapida

Capitulo 28 -- Apendice A: Referencia Rapida

Palavras-chave (39)

```

1 module import pub internal fn struct enum impl trait
2   let mut self Self if elif elseif else unless while
3   do for in of loop match switch case return break
4   continue true false async await

```

Precedencia de Operadores

Nivel	Operadores	Associatividade
1	() f() x.y x[i]	Esq.
2	- ! (unarios)	Dir.
3	* / %	Esq.
4	+ -	Esq.
5	< > <= >=	Esq.
6	== !=	Esq.
7	&&	Esq.
8		Esq.

Visibilidade

Modificador	Acessivel de
(padrao)	Apenas no modulo atual
pub	Qualquer codigo
internal	Mesmo pacote/projeto

Formas de import

```

1 import std.io; // qualificada
2 import std.math as math; // alias
3 import { println } from std.io; // por nome
4 pub import { println } from std.io; // re-export

```

Erros Comuns

Erro	Causa	Solucao
type mismatch	Operacao mista int+float	Use int_to_float() ou float_to_int()
non-exhaustive match	Casos faltando no match	Adicione _ => ou os casos faltantes
break outside loop	break fora de loop	Mova para dentro de while/for/loop
module declaration missing	Arquivo sem 'module nome;'	Adicione 'module nome;' no topo
main not found	Sem ponto de entrada	Adicione 'pub fn main() -> int { return 0; }'
undefined variable	Uso fora do escopo	Garanta que 'let' esta no escopo correto
field not found	Campo inexistente	Verifique o nome do campo na struct

PARTE - CAPÍTULO 29

Apendice B: Gramatica e Mapa de Arquivos

Capitulo 29 -- Apendice B: Gramatica e Mapa de Arquivos

Gramatica Informal (EBNF)

```

1  programa      = declaracao_modulo? import* declaracao* ;
2  declaracao_modulo = "module" IDENT ";" ;
3  import        = "import" caminho_modulo ";"
4                | "import" caminho_modulo "as" IDENT ";"
5                | "import" "{" IDENT ("," IDENT)* "}" "from" caminho_modulo ";"
6                | "pub" "import" "{" IDENT ("," IDENT)* "}" "from" caminho_modulo ";";
7  declaracao    = decl_fn | decl_struct | decl_enum | decl_impl | decl_trait ;
8  decl_fn       = visib? "fn" IDENT genericos? "(" params? ")" ("->" tipo)? bloco ;
9  decl_struct   = visib? "struct" IDENT genericos? "{" campo* "}" ;
10 decl_enum     = visib? "enum" IDENT genericos? "{" variante* "}" ;
11 variante      = IDENT                                (* unit *)
12                | IDENT "(" tipo ("," tipo)* ")"      (* tupla *)
13                | IDENT "{" campo* "}"                (* struct *);
14 decl_impl      = "impl" genericos? IDENT "{" metodo* "}" ;
15 decl_trait     = visib? "trait" IDENT ("," IDENT)? "{" assinatura* "}" ;
16 bloco          = "{" stmt* expr? "}" ;
17 stmt           = decl_let | atribuicao | retorno | expr ";"
18                | laço | condicional | match_stmt ;
19 decl_let       = "let" "mut"? IDENT (":" tipo)? "=" expr ";" ;
20 expr           = literal | IDENT | binario | unario | chamada
21                | met_call | f_string | array_lit | tuple_lit
22                | "if" expr bloco ("elif" expr bloco)* ("else" bloco)?
23                | "unless" expr bloco ("else" bloco)?
24                | "match" expr "{" braco* "}" | closure ;
25 tipo           = "int" | "float" | "bool" | "string" | "char"
26                | IDENT | IDENT "<" tipo ("," tipo)* ">"
27                | "[" tipo "]" | "(" tipo ("," tipo)* ")" ;
28 visib          = "pub" | "internal" ;

```

Mapa de Arquivos do Repositorio

1	compiler/src/lexer/	Lexer (tokens, spans)
2	compiler/src/parser/	Parser (recursivo), ModuleLoader (cache)
3	compiler/src/ast/	Arvore sintatica
4	compiler/src/semantic/	Analise semantica multipassada
5	compiler/src/lint/	Regras de lint
6	compiler/src/pipeline.rs	CompilationPipeline + trait BackendDriver
7	midend/src/ir.rs	IR SSA (SIR) -- instrucoes, blocos, tipos
8	midend/src/lowering.rs	AST -> IR (rebaixamento)
9	midend/src/autodiff.rs	AutodiffStep (no nativo de autodiff)
10	midend/src/tensor_graph.rs	TensorGraph (segunda IR)
11	midend/src/passes/	Passes de otimizacao/verificacao
12	backend/src/codegen.rs	Cranelift JIT
13	backend/src/aot.rs	Cranelift AOT (COFF/ELF/Mach-O)
14	runtime/src/memory/	HybridMemory (GC + manual)
15	runtime/src/ffi.rs	Host-call ABI (SpectraHostCallContext)
16	runtime/src/reactor/	Reactor async (epoll/IOCP/kqueue)
17	runtime/src/api/	Contrato spectra.api (211 calls)
18	runtime/src/stdlib/	Registro de host functions da stdlib
19	tools/spectra-cli/	Binario 'spectralang'
20	tools/spectra-lsp/	LSP (14 capacidades)
21	tools/spectra-interop/	Interop (Python, etc.)
22	packages/spectra-api/	Implementacao spectra.api
23	tests/validation/	172 arquivos .spectra (regressao)
24	tests/errors/	73 arquivos .spectra (negativos)
25	examples/ai/	21 exemplos de IA/ML
26	examples/api/	5 exemplos de API

Apendice C: Stdlib Completa e Roadmap

Todos os Modulos stdlib

Modulo	Funcoes principais
std.io	print, println, eprint, eprintln, read_line, input, flush
std.string	len, contains, to_upper, to_lower, trim, starts_with, ends_with, concat, repeat_str, char_at, substring, replace, index_of, split_first, s
std.math	abs, min, max, clamp, sign, gcd, lcm, abs_f, sqrt_f, pow_f, floor_f, ceil_f, round_f, sin_f, cos_f, tan_f, atan2_f, log_f, log2_f, log10_f, i
std.convert	int_to_string, float_to_string, bool_to_string, string_to_int, string_to_float, string_to_int_or, string_to_bool, int_to_float, float_to_in
std.collections	list_new, list_push, list_pop, list_pop_front, list_len, list_get, list_set, list_insert_at, list_remove_at, list_contains, list_index_of, list_so
std.random	random_seed, random_int, random_float, random_bool
std.fs	fs_read, fs_write, fs_append, fs_exists, fs_remove
std.env	env_get, env_set, env_args_count, env_arg
std.option	is_some, is_none, option_unwrap, option_unwrap_or
std.result	is_ok, is_err, result_unwrap, result_unwrap_or, result_unwrap_err
std.char	is_alpha, is_digit_char, is_whitespace_char, is_upper_char, is_lower_char, is_alphanumeric, to_upper_char, to_lower_char
std.time	time_now_millis, time_now_secs, monotonic_millis, monotonic_nanos, sleep_ms, sleep, duration_ms, duration_secs, duration_mill
std.range	create, len, at, eq, start, end, is_inclusive
std.tensor	Ver Capítulos 5 e 20 para referencia completa (50+ funcoes)
std.ml	Ver Capitulo 21 para referencia completa (50+ funcoes)
std.concurrent	task_spawn, task_join, channel_new, channel_send, channel_rcv, channel_close, counter_new, counter_add, counter_get, pipelin
std.serve	server_new, server_warmup, server_enqueue, server_process_batch, server_result, server_set_input_policy, server_set_output_poli

Flags do CLI -- Resumo Completo

Flag	Descricao
compile	Compila modulos (padrao)
check	Verifica tipos sem gerar codigo
run	Compila e executa via JIT

Flag	Descrição
lint	Executa verificações de lint
fmt	Formata arquivos fonte
repl	Inicia o REPL interativo
new	Cria um novo projeto
-O0 / --no-optimize	Desativa otimizações
-O1	Otimizações básicas
-O2	Otimizações moderadas (padrão)
-O3	Todas as otimizações
--dump-ast	Exibe a AST para debug
--dump-ir	Exibe o IR para debug
--timings / -T	Métricas de compilação
--summary	Sumário do pipeline por módulo
--verbose / -v	Detalhes adicionais do build
--emit-object <out>	Gera arquivo objeto AOT
--emit-exe <out>	Gera executável AOT
--json	Diagnósticos em JSON
--sarif	Diagnósticos SARIF 2.1.0 (GitHub Code Scanning)
--lint	Ativa verificações de lint
--allow <rule>	Suprime uma regra de lint
--deny <rule>	Eleva regra de lint a erro

Fases do Roadmap

Fase	Descrição	Status
0-2	Núcleo da linguagem (lexer, parser, AST, semântica)	Complete
3-7	Tensors, kernels, autodiff, ML framework, dispositivos	Complete
8-13	Interop, package manager, tooling, testes	Complete
14-20	IR, tensor graph, GPU, datasets, ONNX, RAG, avaliação	Complete
21	Async/await core e reator	Complete
22	API foundation (HTTP/1.1, routing, middleware)	Complete
23	Middleware e segurança avançada	In progress
24	Recursos avançados de API (HTTP/2, HTTP/3, SSE, GraphQL)	In progress
25	Persistência e database drivers	In progress
26	API tooling e DX	Not started
27	Observabilidade e operações	In progress

Fase	Descricao	Status
28	API conformance v1.0 e release	Not started
29-31	Exact-width numerics, string/buffer optimizations	In progress

Documentos de Apoio no Repositorio

docs/reference/01-06: Referencia completa da linguagem (6 partes). docs/book/01-10: The Spectra Book. docs/AI-AGENT-REFERENCE.md: Referencia para agentes AI. docs/language-feature-maturity.md: Matriz de maturidade. docs/api/*: 24 docs de referencia da API. docs/adr/*: Architecture Decision Records (0001-0011). docs/diagnostics/error-code-reference.md: Referencia de codigos de erro. docs/ARCHITECTURE.md: Arquitetura geral. docs/production-ai-implementation-plan.md: Plano estrategico. roadmap/roadmap.toml: Roadmap estruturado (machine-readable). docs/roadmap-backlog.md: Backlog humano.

Fim do Guia

Este guia cobre a linguagem SpectraLang versao 0.3.0 de forma abrangente, desde os fundamentos absolutos da programacao ate os recursos mais avancados da linguagem e sua toolchain. Para contribuicoes, bugs ou duvidas, consulte o repositorio do projeto. O codigo-fonte deste ebook esta em presentation/spectra_ebook_pdf.py. Para regenera-lo, execute `python presentation/spectra_ebook_pdf.py`.

```
1 python presentation/spectra_ebook_pdf.py
2 # Gera: presentation/spectra_ebook.pdf
```